

MOTOWYCENA.PL

Plan przebudowy i optymalizacji SEO

Klient:	Rafal Pelczar - Rzecznawca Techniki Samochodowej
Certyfikat:	RS001771
Domena:	https://www.motowycena.pl
Stack docelowy:	Astro 5 + React 19
Czas wdrożenia:	3-4 tygodnie
Ryzyko utraty pozycji SEO:	ponizej 5%
Data dokumentu:	Maj 2026

Pelna dokumentacja techniczna i strategiczna projektu

Spis treści

CZESC I - DLA KLIENTA

1. Wytłumaczenie po ludzku
2. Podsumowanie wykonawcze

CZESC II - ANALIZA TECHNICZNA

3. Audyt SEO obecnej strony
4. Wybór stacka technologicznego
5. Mapa URL (strategia 1:1)

CZESC III - WDROZENIE

6. Plan migracji - krok po kroku
7. Checklist migracji
8. Konkretnie ulepszenia SEO

CZESC IV - SETUP TECHNICZNY

9. Turnstile + Resend - integracja

CZESC V - ENCYKLOPEDIA TECHNICZNA

10. Pojęcia + definicje + kod (40+ tematów)
 - A. Framework i frontend (Astro, React, TS, Tailwind)
 - B. SEO techniczne (meta, JSON-LD, OG, sitemap)
 - C. Performance i Core Web Vitals
 - D. Formularze i backend (PE, PRG, Turnstile, Resend)
 - E. Bezpieczeństwo i prawo (HTTPS, HSTS, RODO, CSP)
 - F. Strategia URL i linkowanie
 - G. Hosting i deploy (Cloudflare, Edge, CI/CD)
 - H. Narzędzia pomiarowe (Lighthouse, RUM)
 - I. Content management (Markdown, MDX, Headless CMS)
 - J. Wersjowanie i package management

DODATKI

11. Słowniczek pojęć

CZESC I

Dla klienta

1. Wytlumaczenie po ludzku

Co przebudowujemy w Pana stronie - wytłumaczone po ludzku

Dla: Rafał Pelczar, motowycena.pl Data: maj 2026

1. Co Pan ma teraz

Pana strona **motowycena.pl** to taki sklep w internecie. Zbudowany jest na popularnym "zestawie klocków" (WordPress). Działa, ale ma kilka problemów których Pan nie widzi gołym okiem.

Wyobraź sobie sklep przy ulicy. Drzwi się otwierają, klient wejdzie. Ale:

- **Brak szyldu z opisem** - ludzie idący ulicą nie wiedzą co Pan sprzedaje
- **Brak wizytówki przy kasie** - Google nie wie czym Pan się zajmuje, więc Pana nie poleca
- **Wystawa się powoli wczytuje** - klienci uciekają zanim wejdą
- **Na półkach brakuje napisów** - ktoś wchodzi i nie wie gdzie szukać

Czy strona działa? Tak. Czy przynosi tyle klientów ile mogłaby? **Nie. Tracimy bardzo dużo.**

2. Co my robimy

Budujemy **nowy sklep pod tym samym adresem**. Klienci którzy znali stary adres trafiają na nowy bez żadnej zmiany. Google też - więc Pana pozycje w wyszukiwarce nie znikają.

Ale wszystko inne jest nowe:

- Nowoczesne, szybkie technologie - strona wczytuje się **5-10x szybciej**
- Każda Pana usługa ma teraz "podpis" w Google
- Google dostaje pełną wizytówkę - kto, gdzie, ile, kiedy, certyfikat
- Ładny wygląd na komórce
- Pływający przycisk WhatsApp - klient zawsze ma 1 klik do kontaktu
- Darmowy serwer (Cloudflare) - zamiast płacić rocznie za hosting, dostaje Pan to za 0 zł

3. Czy mi Pan czegoś nie zepsuje?

Nie. I właśnie to jest najważniejsza decyzja jaką podjęliśmy.

Wyobraź sobie remont domu. Można:

- **Wariant A:** wyburzyć dom i zbudować nowy - tracisz adres, klienci się gubią
- **Wariant B:** zostawić te same drzwi w tych samych miejscach, ale wszystko za nimi przerobić - klienci wchodzą tak jak wchodzili, tylko w środku jest pięknie

Robimy wariant B. Wszystkie 15 adresów Pana podstron zostaje identyczne:

- [motowycena.pl/wycena-celno-skarbowa/](#) <- zostaje
- [motowycena.pl/biegly-sadowy/](#) <- zostaje

- motowycena.pl/realizacje/ <- zostaje
- ...i 12 innych

Google nawet nie zauważy że coś przebudowaliśmy. Po prostu Pana ranking **idzie do góry**, nie w dół.

Ryzyko utraty pozycji: poniżej 5%.

4. Co konkretnie Pan zyska

Po przebudowie, w ciągu 3-12 miesięcy:

Co	Dziś	Po przebudowie
Szybkość strony	~3-4 sekundy	< 1 sekunda
Widoczność w Google	Słaba	2-3x lepsza
Klikalność z Google	Niska (brak opisu)	2x wyższa
Pozycja w wyszukiwarce	Średnia	+10 pozycji w górę
Zapytania (formularz/telefon)	Tyle co teraz	+50-100% więcej

W praktyce: dziś strona przynosi powiedzmy 10 zapytań miesięcznie. Po przebudowie - realnie **15-25**. **Ta sama strona, ten sam adres, tylko lepiej zaprojektowana.**

5. Co dziś zrobiliśmy

- 1 **Audyt** - sprawdziłem co Pan ma, czego brakuje
- 2 **Wybór technologii** - Astro 5 + React 19 (najszybsze, najlepsze pod Google)
- 3 **15 nowych podstron** - każda z tytułem, opisem, wizytówką, FAQ, kontaktem
- 4 **Strona dla Ostrowa Wielkopolskiego** - Pana "domowe miasto" (powiat ostrowski, gmina Odolanów). Jak ktoś szuka "rzeczoznawca samochodowy Ostrów" - znajdzie Pana
- 5 **Formularz który zawsze działa** - z włączonym i wyłączonym JavaScript, z ochroną przed spamem
- 6 **Licznik odwiedzin** - bez ciastek, zgodny z RODO
- 7 **Struktura do edycji treści** - zaczynamy od plików tekstowych, w przyszłości dodamy panel jak w WordPressie

6. Co dalej

Trzy pytania do Pana:

1. Czy ruszamy? W 3-4 tygodnie mam Panu nową stronę gotową.

2. Co Pan chce dodać poza standardem? (mocno polecam [OK])

- [OK] **Blog SEO** (4-8 artykułów / mc -> więcej ruchu z Google)
- [OK] **Kalkulator wstępnej wyceny** (klient wpisuje markę/rok -> cena orientacyjna)
- [OK] **Podstrony lokalne** (Kalisz, Wrocław, Poznań, Warszawa)

Później:

- Płatności online (BLIK, karta)

- Strefa klienta (pobieranie raportów PDF)
- Wersja po angielsku/niemiecku

3. Czego Pan potrzebuje ode mnie?

- Dostęp do Google Search Console (jeśli Pan ma)
- Hasło do panelu domeny
- Logo SVG (jeśli ma)
- Zdjęcia z realizacji
- 2-3 opinie klientów

7. Najczęstsze pytania

P: Ile to kosztuje? Praca jednorazowa. Hosting na Cloudflare jest darmowy. Koszty stałe:

- Domena ~50 zł/rok (już Pan ma)
- Mail do formularza - darmowe (Resend)
- Panel CMS - darmowy (do 2 osób)

Łącznie po wdrożeniu **~50 zł/rok** zamiast obecnych 200-500 zł/rok za WordPress.

P: A jak coś się popsuje? Stara strona zostaje w archiwum 90 dni. **15 minut** i przełączamy z powrotem. Nikt nie zauważy.

P: Czy stracę klientów którzy zapisali stronę w zakładkach? Nie. Adresy są identyczne.

P: Czy moje zdjęcia, teksty, certyfikaty zostają? Tak. Przenosimy wszystko, dodajemy brakujące rzeczy.

P: Czy będę mógł sam edytować teksty? Tak. Najpierw przez prosty edytor (uczę Pana 30 min). Później dorzucamy wizualny panel.

P: Co jeśli Google mnie zepchnie? Google **lubi** szybsze, lepiej opisane strony. Spodziewamy się wzrostu. Jeśli przez 4 tygodnie widzimy spadek - mam plan ratunkowy.

Słowniczek (jeśli ktoś zapyta)

Słowo techniczne	Po ludzku
SEO	Co robi że Google Pana znajduje
Meta description	Krótki opis pod tytułem w Google
Schema markup	Wizytówka strony którą czyta Google
Astro / React	Nowoczesne klocki do budowy strony
Cloudflare	Darmowy "supermarket" hostingu
Sitemap	Spis treści strony dla Google
Trailing slash	Ukośnik na końcu adresu (zostawiamy taki sam jak ma Pan teraz)
Schema LocalBusiness	"Pan = lokalna firma" - Google rozumie i poleca

Core Web Vitals	Ocena jakości strony przez Google (Pan dostanie 95+/100)
OG image	Ładny obrazek gdy ktoś udostępni link na Facebooku
Progressive enhancement	Formularz działa zawsze, nawet bez JavaScript
Cloudflare Turnstile	Bramkarz przeciwko spammerskim robotom
Resend	Listonosz dla maili z formularza
Honeypot	Niewidzialna pułapka na boty
Content collection	Folder z plikami treści (dla łatwej edycji)
TinaCMS	Wizualny panel edycji - jak w WordPressie ale szybszy
Cloudflare Pages	Darmowy hosting na komputerach Cloudflare
301 redirect	Przekierowanie ze starego adresu na nowy (my nie potrzebujemy bo zostawiamy stare adresy)
Cache	Pamięć podręczna - żeby strona była szybsza za drugim razem
HTTPS	Zielona kłódka - bezpieczne połączenie
LCP / CLS / INP	Pomiary szybkości używane przez Google do oceny strony

Wszystkie te rzeczy są **już zrobione w Pana wersji** - Pan nie musi się tym zajmować, ja to konfiguruję.

2. Podsumowanie wykonawcze

Motowycena.pl - Plan przebudowy (executive summary)

Dla: Rafał Pelczar, Motowycena **Cel:** Nowa, szybsza, lepiej widoczna w Google strona - **bez utraty obecnych pozycji**

W skrócie (60 sekund czytania)

Twoja obecna strona oparta jest na WordPress + Elementor (Astra theme). Działa, ale ma **poważne braki SEO** (brak meta opisów, brak schema, brak Open Graph) które ograniczają widoczność w Google.

Nasza propozycja: przepisanie strony na **Astro 5 + React 19** - nowoczesny, błyskawicznie szybki stack zoptymalizowany pod Google.

Najważniejsze:

- [OK] **Wszystkie 15 obecnych adresów URL pozostaje BEZ ZMIAN** -> zero ryzyka utraty pozycji
- [OK] **Pełne SEO od zera:** meta tagi, schema markup, Open Graph, FAQ structured data
- [OK] **Strona 5-10x szybsza** (Lighthouse 95+ vs obecnie ~60)
- [OK] **Nowoczesny design** + element konwersyjne (kalkulator, formularze, WhatsApp)
- [OK] **Hosting: 0 zł** (Cloudflare Pages - darmowy, globalny CDN)

Co konkretnie poprawiamy

Element	Dziś	Po migracji
Meta description	brak na 15/15 stron	unikalna na każdej
Schema markup	brak	LocalBusiness, Service, FAQ, Person
Open Graph (FB/LinkedIn share)	brak	pełny zestaw
H1 nagłówki	brak na 8/15 stron	unikalny H1 na każdej
Lighthouse Performance	~60	95+
LCP (szybkość ładowania)	~3-4s	<1s
Core Web Vitals	nieznane	zielone
Mobile UX	OK	sticky WhatsApp + kalkulator
Formularz	tylko email	+ walidacja + anti-spam + auto-odpowiedź
Hosting	LiteSpeed (Polska)	Cloudflare globalny CDN

Jak zabezpieczamy SEO przed utratą

- 1 **Zachowujemy 100% adresów URL** - żadnych przekierowań
- 2 **Budujemy na osobnej subdomenie** (staging) - Google nie widzi w czasie budowy
- 3 **Backup obecnej strony** przed jakimkolwiek dotknięciem
- 4 **Wdrożenie w nocy** (2:00) - minimum ruchu
- 5 **Plan B: rollback w 15 minut** jeśli coś pójdzie nie tak
- 6 **Monitoring 24/7 przez 30 dni** po wdrożeniu

Realne ryzyko utraty pozycji: <5% (znacznie mniejsze niż typowa migracja, bo obecne SEO i tak jest słabe - jest co poprawiać, mało co tracić).

Timeline

Tydzień	Co się dzieje	Twoje zaangażowanie
0	Dostępny, backupy, audyt	30 min - przekazanie haseł
1	Setup projektu, design systemu	-
2	Budowa wszystkich 15 podstron	2-3h - konsultacja treści
3	Testy, optymalizacja, akceptacja	2h - akceptacja na staging
3 (noc)	Deployment (przełączenie domeny)	- (śpisz)
4-12	Monitoring i optymalizacja	- (raport co tydzień)

Razem: 3-4 tygodnie do wdrożenia, 90 dni pełnej stabilizacji.

Co dostajesz w cenie

- [OK] Pełna analiza i audyt SEO obecnej strony
- [OK] Nowa strona zbudowana od zera (Astro 5 + React 19)
- [OK] Wszystkie 15 obecnych podstron + 5 nowych ([/o-mnie/](#) , [/faq/](#) , [/cennik/](#) , [/opinie/](#) , [/kalkulator/](#))
- [OK] Schema markup na każdej stronie
- [OK] Sitemap.xml + robots.txt + manifest
- [OK] Formularz kontaktowy z anti-spamem (Cloudflare Turnstile)
- [OK] Pływający WhatsApp button
- [OK] Cookie banner zgodny z RODO
- [OK] Setup Cloudflare Pages (hosting darmowy)
- [OK] Setup Cloudflare Web Analytics (darmowe, RODO-friendly)
- [OK] Konfiguracja Google Search Console
- [OK] Submit sitemap do Google + Bing
- [OK] Backup obecnej strony (90 dni archiwum)

- [OK] Dokumentacja jak edytować treści w przyszłości
- [OK] 30 dni wsparcia po wdrożeniu

Co możemy dodać później (poza zakresem)

- **Kalkulator szacunkowy** wyceny online (konwersja +30%)
- **Blog SEO** (2-4 artykuły/mc -> ruch organiczny w 3-6 mies.)
- **Strefa klienta** (logowanie, pobieranie ekspertyz PDF)
- **Płatności online** (Stripe/Przelewy24 - BLIK)
- **Booking online** (Cal.com - rezerwacja terminu)
- **Wersja angielska/niemiecka** (emigranci wracający z UK/DE)
- **Podstrony lokalne** (Rzeczoznawca Poznań/Wrocław/Warszawa...)

Decyzja

Mam **dwa pytania** do Ciebie:

- 1 **Czy ruszamy z tym planem?** Jeśli tak - proszę o dostęp do dokumentu `docs/05-CHECKLIST.md`.
- 1 **Czy oprócz core'u chcesz dodać coś z listy "później" już teraz?** (kalkulator i blog mocno polecam - to konwersja i ruch).

Po Twoim "tak" - startuję ze scaffoldem projektu i w tydzień masz pierwszą wersję na staging do oglądania.

Pełna dokumentacja w folderze `docs/` :

- [01-AUDYT-SEO.md](#) - co znalazłem
- [02-STACK-DECYZJA.md](#) - dlaczego Astro
- [03-URL-MAPPING.md](#) - mapa adresów
- [04-PAN-MIGRACJI.md](#) - dzień po dniu
- [05-CHECKLIST.md](#) - do odhaczania
- [06-SEO-IMPROVEMENTS.md](#) - co poprawiamy w SEO

CZESC II

Analiza techniczna

3. Audyt SEO obecnej strony

Audyt SEO obecnej strony motowycena.pl

Data audytu: 2026-05-20

Podstawowe informacje techniczne

Element	Wartość
CMS	WordPress 6.9.4
Motyw	Astra
Page builder	Elementor 4.0.8
Formularze	WPForms Lite
Dodatki	Happy Elementor Addons, Astra Sites
Serwer	LiteSpeed
HTTP	HTTP/2 + HTTP/3 (QUIC) [OK]
HTTPS	Tak [OK]
<code>wp-json</code> REST API	Aktywne (publiczne)

Sitemap i indeksacja

Lokalizacja sitemap: `https://www.motowycena.pl/wp-sitemap.xml` (natywna WP, nie z pluginu SEO)

Liczba URL-i w sitemapie: 15 + 1 user (autor)

Status `robots.txt` :

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php
Sitemap: https://www.motowycena.pl/wp-sitemap.xml
```

[OK] Poprawny, niczego nie blokuje przed Google.

Pełna lista URL (15 stron)

#	URL	Ostatnia modyfikacja
1	<code>/</code> (home)	2022-01-29
2	<code>/realizacje/</code>	2022-01-10
3	<code>/kontakt/</code>	2022-01-29
4	<code>/wycena-kosztow-naprawy/</code>	2022-01-11
5	<code>/wycena-wartosci-pojazdu/</code>	2022-01-11

6	/opis-stanu-technicznego/	2023-03-29
7	/wycena-celno-skarbowa/	2022-01-11
8	/doradztwo-zakupowe/	2022-01-11
9	/polityka-prywatnosci/	2023-01-02
10	/pojazdy-zabytkowe-2/ [!]	2022-03-01
11	/zmiany-konstrukcyjne/	2022-12-09
12	/pomoc-prawna/	2022-12-12
13	/polisy-ubezpieczeniowe/	2023-03-29
14	/biegly-skarbowy/	2023-11-26
15	/biegly-sadowy/	2023-11-26

[!] **Uwaga** dla /pojazdy-zabytkowe-2/ : sufix -2 sugeruje że oryginał był pod /pojazdy-zabytkowe/ i został usunięty/przeniesiony. **Nie zmieniamy** tego URL w migracji - Google już go zna.

[!] **Ostatnia aktualizacja treści 2023-11-26.** Strona nie była aktualizowana od ~2.5 roku. Świeżość treści to czynnik rankingowy.

Krytyczne braki SEO (zmierzone na wszystkich 15 stronach)

Brak meta description (0/15)

Wszystkie strony mają puste `<meta name="description">` . To oznacza że:

- Google sam losuje fragment treści na SERP (często bezsensowny)
- CTR z wyników wyszukiwania jest niższy o ~30%
- Tracimy okazję do zachęcenia kliknięcia

Brak Open Graph (0/15)

Brak tagów `og:title` , `og:description` , `og:image` . Przy udostępnianiu strony na Facebooku/LinkedIn/WhatsApp pokazuje się surowy link bez ładnego podglądu.

Brak Schema.org / JSON-LD (0/15)

Brak structured data dla:

- `LocalBusiness` (krytyczne dla local SEO)
- `Service` (każda usługa powinna mieć schemę)
- `Person` (Rafał Pelczar jako rzeczoznawca)
- `FAQPage` (brak FAQ w ogóle)
- `Review` / `AggregateRating` (brak opinii)
- `BreadcrumbList` (nawigacja okruszkowa)

To ogromna strata - Google bez schemy nie wie co opisuje strona.

Brak H1 na 8/15 stronach

Strony bez H1:

- /kontakt/
- /wycena-kosztow-naprawy/
- /wycena-wartosci-pojazdu/
- /opis-stanu-technicznego/
- /wycena-celno-skarbowa/
- /doradztwo-zakupowe/
- /zmiany-konstrukcyjne/

H1 to **podstawowy sygnał** dla Google o czym jest strona. Brak H1 = utracona pozycja na main keyword.

Title tagi - do poprawy

URL	Obecny title	Problem
/	Motowycena	10 znaków, brak USP, brak lokalizacji, brak słów kluczowych
Wszystkie podstrony	[Nazwa] - Motowycena	Generic format, brak modyfikatorów (cena, szybko, region)

Co działa (do zachowania)

[OK] **Canonical tags** - poprawne na każdej stronie [OK] **HTTPS** - bez błędów certyfikatu [OK] **URL slug** - czyste, z keywords (/wycena-celno-skarbowa/ - idealny) [OK] **Mobile viewport** meta - obecny [OK] **HTTP/3** - nowoczesny protokół [OK] **Sitemap XML** - generowany automatycznie

Wnioski strategiczne

Obecne SEO jest na poziomie podstawowym (3/10). To paradoksalnie **DOBRA wiadomość** dla migracji:

- 1 Klient ma niewiele do stracenia
- 2 Każdy element który dodamy = czysty zysk
- 3 Po migracji można spodziewać się **2-3x lepszej widoczności w 3-6 miesięcy**
- 4 Stara strona prawdopodobnie nie rankuje wysoko (przez braki SEO), więc nie ma "wysokich pozycji do utraty"

Najważniejsze zyski po migracji

- ☐ Meta description na każdej stronie -> +30% CTR
- ☐ Schema markup (LocalBusiness, Service, FAQPage) -> rich snippets w Google
- ☐ Open Graph -> ładne karty przy udostępnianiu
- ☐ H1 na każdej stronie z głównym keyword -> lepszy ranking
- ☐ Świeże daty modyfikacji -> sygnał świeżości
- ☐ Core Web Vitals 95+/100 (Astro) -> ranking boost
- ☐ FAQ + breadcrumbs -> rich snippets

Co zachowujemy 1:1 (bezwzględnie)

- Wszystkie 15 URL-i (bez wyjątku, łącznie z -2)
- Canonical tags
- Strukturę informacji (te same sekcje, te same usługi)
- Numer certyfikatu RS001771, NIP, dane kontaktowe

4. Wybór stacka technologicznego

Wybór stacka technologicznego

Decyzja: Astro 5 + React 19 (islands)

Po analizie potrzeb biznesowych klienta i charakterystyki strony, najlepszym wyborem jest **Astro 5 z React-owymi wyspami interaktywności**.

Dlaczego nie Next.js 16

Next.js 16 to świetny framework, ale dla strony motowycena.pl jest **overkill**:

Aspekt	Next.js 16	Astro 5
JavaScript w przeglądarce	80-120 KB minimum	0 KB domyślnie
Lighthouse Performance (out-of-box)	75-90	95-100
LCP (Largest Contentful Paint)	1.5-3s	<1s
Core Web Vitals (sygnał Google)	Dobre	Najlepsze
Złożoność dla 15 stron	Wysoka	Niska
Czas budowy	Wolniejszy	Szybszy
Hosting	Wymaga Node (Vercel)	Statyczny (każdy CDN)
Cena hostingu	\$0-20/mc	\$0 (Cloudflare Pages)
Krzywa uczenia dla utrzymującego	Stroma	Łagodna
SEO out-of-the-box	Dobre	Najlepsze

Dlaczego Astro 5 jest IDEALNY dla tego projektu

1. Strona jest w 95% statyczna

Treści typu "Wycena celno-skarbowa", "Pojazdy zabytkowe", "Pomoc prawna" **nie zmieniają się dynamicznie**. Astro generuje czysty HTML w build time -> Google to uwielbia.

2. Islands Architecture = najlepsze z obu światów

Tylko te elementy które wymagają interaktywności renderują się jako React:

- Kalkulator wyceny (React + react-hook-form)
- Formularz kontaktowy (React + zod validation)
- Pływający przycisk WhatsApp
- Cookie banner
- Slider opinii klientów

Reszta strony to czysty, błyskawiczny HTML/CSS. **Każdy bajt JS jest świadomą decyzją.**

3. Najlepsze Core Web Vitals na rynku

Google od 2021 oficjalnie używa Core Web Vitals jako czynnika rankingowego. Astro wygrywa tu z każdym frameworkiem React-based.

4. Content Collections - idealne pod blog

Astro ma wbudowane Content Collections (MDX + TypeScript). Gdy klient zechce blog, dodajemy folder `src/content/blog/` i każdy plik `.mdx` to artykuł. Z auto-generowanym sitemap, OG image, schema BlogPosting.

5. View Transitions API

Astro 5 wspiera natywnie View Transitions - płynne przejścia między podstronami bez SPA. Wygląda jak nowoczesny app, ale to nadal statyczny HTML (SEO-friendly).

6. Można wpinać dowolny framework

Jeśli klient kiedyś zechce panel klienta lub bardziej interaktywne komponenty, można dorzucić **Solid**, **Svelte**, **Vue** lub **React** - Astro obsługuje wszystko.

7. Łatwy upgrade do SSR jeśli potrzeba

Astro 5 ma tryb hybrydowy - możesz na 1 stronie włączyć SSR (np. dla strefy klienta z logowaniem), reszta zostaje statyczna.

Pełny stack

Frontend

Warstwa	Technologia	Wersja	Dlaczego
Framework	Astro	5.x	Najlepsze SEO/perf dla content-driven
UI w islands	React	19	Najnowszy, klient prosił o "nowszy React"
Język	TypeScript	5.x	Type safety, łatwiejsze utrzymanie
Styling	Tailwind CSS	4.x	Utility-first, mały bundle
Komponenty UI	shadcn/ui	latest	Copy-paste komponenty React
Ikony	lucide-react	latest	Lekkie SVG icons
Animacje	Motion (dawniej Framer Motion)	latest	Tylko gdzie potrzeba
Fonty	Geist / Inter (self-hosted)	latest	Bez Google Fonts (RODO + perf)

Formularze i walidacja

- **react-hook-form** + **zod** w islands
- Server endpoint w Astro (`src/pages/api/contact.ts`)

- Email: **Resend** (3000 maili/mc darmowo) lub natywny SMTP klienta
- Anti-spam: **Cloudflare Turnstile** (lepszy od reCAPTCHA, darmowy)

SEO

- **@astrojs/sitemap** - auto-generowany sitemap.xml
- **astro-seo** lub własna implementacja `<SEO />` komponentu
- Schema.org JSON-LD jako komponent astro
- OG images: **@vercel/og** lub generator w Astro

Optymalizacja obrazów

- **@astrojs/image** - auto WebP/AVIF, lazy loading, responsive
- Lub **Cloudflare Images** (\$5/mc, jeśli więcej)

Analytics i monitoring

- **Cloudflare Web Analytics** (darmowe, bez cookies, RODO-friendly)
- Lub **Plausible** (EUR9/mc, RODO-friendly)
- **Google Search Console** (must-have do śledzenia migracji)
- ~~Google Analytics~~ - dyskusyjne pod RODO, opcjonalne

Hosting i deploy

- **Cloudflare Pages** - darmowy plan, unlimited bandwidth, globalny CDN, HTTPS
- Alternatywa: **Vercel** (darmowy hobby plan) lub **Netlify**
- Build z **GitHub Actions** - automatyczne deploje z gita

Edytowanie treści (CMS) - opcja

Klient w obecnej wersji edytuje przez Elementor. W Astro mamy opcje:

Opcja A: Markdown w gicie (najprostsze, bezpłatne)

- Treści w `src/content/*.md` lub `.mdx`
- Edycja przez GitHub web editor lub VS Code
- Wadą: wymaga gita

Opcja B: TinaCMS (darmowy do 2 użytkowników)

- Wizualny edytor w przeglądarce
- Backend: GitHub
- Edycja "in-place" jak w Elementor

Opcja C: Sanity / Strapi / Payload CMS (headless)

- Pełny panel admina
- Bezpłatny tier dla małych firm
- Trochę więcej konfiguracji

Rekomendacja: zaczynamy od **B (TinaCMS)** - klient czuje że "edytuje wizualnie" jak w Elementor.

Dodatkowe technologie (gdy klient chce więcej)

Funkcja	Stack
Kalkulator wyceny	React + react-hook-form + zod + Tailwind
Płatności online	Stripe + Przelewy24 (BLIK)
Booking online	Cal.com (self-hosted) lub Calendly embed
Live chat	Crisp lub Tidio
WhatsApp button	Własny komponent (lekki)
Strefa klienta	Astro SSR + Clerk/Better Auth
Wielojęzyczność	Astro i18n routing

Podsumowanie

Astro 5 + React 19 daje:

- [OK] "Nowszy React" (wersja 19 - najnowsza)
- [OK] Najlepsze możliwe SEO (Core Web Vitals 95+)
- [OK] Najszybsza strona w branży
- [OK] Niski koszt utrzymania (hosting \$0)
- [OK] Łatwy do rozbudowy o kalkulator, blog, strefę klienta
- [OK] TypeScript = mniej bugów w przyszłości
- [OK] Modern stack = łatwo znaleźć dev w przyszłości

To nie jest decyzja "modnego frameworka" - to **strategiczny wybór** który optymalizuje pod biznes klienta (więcej leadów z Google).

5. Mapa URL - strategia 1:1

Mapa URL - strategia 1:1 (zero ryzyka SEO)

Zasada główna

ZACHOWUJEMY WSZYSTKIE 15 URL-i DOKŁADNIE TAKIE JAKIE SĄ.

To najbezpieczniejsza strategia migracji. Skoro Google już zna te URL-e (z lastmod sprzed 2-4 lat), zmiana czegokolwiek = ryzyko utraty pozycji.

Tabela mapowania (stary URL -> nowy URL)

Stary URL (WordPress)	Nowy URL (Astro)	Zmiana	Komentarz
/	/	-	Strona główna
/realizacje/	/realizacje/	-	Z trailing slash
/kontakt/	/kontakt/	-	
/wycena-kosztow-naprawy/	/wycena-kosztow-naprawy/	-	Główna usługa
/wycena-wartosci-pojazdu/	/wycena-wartosci-pojazdu/	-	Główna usługa
/opis-stanu-technicznego/	/opis-stanu-technicznego/	-	Główna usługa
/wycena-celno-skarbowa/	/wycena-celno-skarbowa/	-	Główna usługa
/doradztwo-zakupowe/	/doradztwo-zakupowe/	-	Główna usługa
/polityka-prywatnosci/	/polityka-prywatnosci/	-	
/pojazdy-zabytkowe-2/ [!]	/pojazdy-zabytkowe-2/	-	NIE ruszamy mimo brzydkiego -2
/zmiany-konstrukcyjne/	/zmiany-konstrukcyjne/	-	
/pomoc-prawna/	/pomoc-prawna/	-	
/polisy-ubezpieczeniowe/	/polisy-ubezpieczeniowe/	-	
/biegly-skarbowy/	/biegly-skarbowy/	-	Dochodowa nisza
/biegly-sadowy/	/biegly-sadowy/	-	Dochodowa nisza

Zmian: 0 Redirectów 301 wymaganych: 0 Ryzyko utraty pozycji z powodu URL: 0%

Konfiguracja Astro pod trailing slash

WordPress używa **trailing slash** (`/kontakt/` z `/` na końcu). Musimy to zachować w Astro:

astro.config.mjs`

```
import { defineConfig } from 'astro/config';

export default defineConfig({
  site: 'https://www.motowycena.pl',
  trailingSlash: 'always',          // <-- KRYTYCZNE!
  build: {
    format: 'directory',          // generuje /kontakt/index.html zamiast /kontakt.html
  },
});
```

Bez tej konfiguracji Astro wygeneruje `/kontakt` (bez slash'a) -> Google dostanie 301 z `/kontakt/` -> `/kontakt` -> utrata sygnałów rankingowych.

Dodatkowe URL-e do dodania (nowe, bez wpływu na SEO)

Po migracji warto dodać te strony (Google je zaindeksuje jako nowe, nie konkurują ze starymi):

Nowy URL	Cel	Priorytet
<code>/blog/</code>	Index bloga (SEO-driven content marketing)	P1
<code>/blog/[slug]/</code>	Pojedyncze artykuły	P1
<code>/faq/</code>	Często zadawane pytania (rich snippet)	P1
<code>/cennik/</code>	Cennik usług (frazy "ile kosztuje wycena")	P1
<code>/o-mnie/</code>	Strona o rzeczoznawcy	P0
<code>/opinie/</code>	Opinie klientów (social proof + schema)	P1
<code>/rzeczoznawca-poznan/</code>	Local SEO	P2
<code>/rzeczoznawca-wroclaw/</code>	Local SEO	P2
<code>/rzeczoznawca-warszawa/</code>	Local SEO	P2
<code>/rzeczoznawca-krakow/</code>	Local SEO	P2
<code>/kalkulator-wyceny/</code>	Kalkulator online (conversion booster)	P0

Trailing slash - test po wdrożeniu

Po deployu **MUSI** zwracać 200 OK (nie 301):

```
# Test każdego URL-a
curl -sI https://www.motowycena.pl/kontakt/ | grep -i "HTTP/"
# Oczekiwane: HTTP/2 200
```

```
# Test bez slash -> powinien 301 NA wersję z slash
curl -sI https://www.motowycena.pl/kontakt | grep -i -E "HTTP|location"
# Oczekiwane: HTTP/2 301 + Location: /kontakt/
```

Backlinki - sprawdź przed migracją

Przed deployem zrób eksport backlinków z:

- Google Search Console -> Linki -> Top linkujące witryny
- Ahrefs Free Backlink Checker (3 linki za darmo)
- Semrush Backlink Analytics (trial)

Każdy URL który ma backlinki MUSI nadal działać. Jeśli zachowujemy 1:1 - ten warunek jest spełniony automatycznie.

Co z wp-admin , wp-login , feed?

Te URL-e WordPress są specyficzne dla CMS i Google ich nie indeksuje (robots.txt blokuje). Po migracji **przestają istnieć** - to bezpieczne, nie wpływa na SEO.

Stare URL-e do **bezpiecznego usunięcia** (Google ich nie zna):

- /wp-admin/
- /wp-login.php
- /feed/
- /wp-json/*
- /?p=123 (numeryczne ID postów - nikt nie linkuje)

Final sitemap dla nowej strony

Po migracji wygenerujemy nowy sitemap zawierający:

- 15 starych URL-i (z aktualizowanym `lastmod`)
- ~10 nowych URL-i (blog, FAQ, cennik, lokalne)

Plik: `https://www.motowycena.pl/sitemap-index.xml` (auto przez `@astrojs/sitemap`)

Po deployu zgłaszamy w GSC: **Sitemaps -> Add new sitemap -> sitemap-index.xml**

CZESC III

Wdrozenie

6. Plan migracji - krok po kroku

Plan migracji - krok po kroku

Założenia

- Czas trwania: **3-4 tygodnie** (zależnie od dostępności klienta do konsultacji treści)
- Downtime: **0 minut** (rolling deployment przez DNS)
- Ryzyko utraty pozycji: **<5%**
- Plan B: pełny rollback w <15 minut

ETAP 0: Przygotowanie (Tydzień 0, dni 1-3)

Wymagane dostępy od klienta

Dostęp	Po co	Krytyczność
Google Search Console (właściciel)	Monitoring migracji	[P0] MUST
Google Analytics (jeśli ma)	Baseline ruchu	[P1] Mocno zalecane
Hosting WordPress (FTP/cPanel)	Backup + 301 fallback	[P0] MUST
DNS (Cloudflare/OVH/inny rejestr)	Przełączenie domeny	[P0] MUST
Logo, zdjęcia, dokumenty firmowe	Treść nowej strony	[P1] Wymagane
Skan certyfikatu RS001771	Wzmocnienie zaufania	[OK] Nice to have

Inwentaryzacja PRZED jakimkolwiek dotykem

Pełny backup WordPress (pliki + DB)
Eksport z GSC: wszystkie URL-e + top 100 fraz
Crawl Screaming Frog (darmowy do 500 URL, mamy 15)
→ eksport: meta, h1, h2, alt, internal links
Screenshot każdej z 15 podstron (referencja wizualna)
Test obecnej szybkości: PageSpeed Insights
→ notuj LCP, FID, CLS, INP
Test mobile-friendly: Google Mobile-Friendly Test
Eksport backlinków (Ahrefs Free / GSC)

Setup nowego środowiska

```
# Repository
git init
npm create astro@latest motowycena-new
cd motowycena-new

# React + Tailwind + integracje
npx astro add react tailwind sitemap mdx

# Dodatkowe
npm install zod react-hook-form lucide-react
npm install -D @types/node
```

Deploy preview na subdomenę

- Setup: `staging.motowycena.pl` (nowy serwer, np. Cloudflare Pages)
- **OBYWIAZKOWO:** robots.txt z `Disallow: /` na staging
- **OBYWIAZKOWO:** `<meta name="robots" content="noindex,nofollow">` na każdej stronie
- **OBYWIAZKOWO:** Basic Auth (login/hasło) - Cloudflare Access za darmo

Bez tych zabezpieczeń Google zaindeksuje staging i powstanie duplicate content.

ETAP 1: Budowa nowej strony (Tygodnie 1-2)

Komponenty bazowe (pierwszy tydzień)

```
src/
├── components/
│   ├── layout/
│   │   ├── Header.astro      # nagłówek z menu + CTA WhatsApp
│   │   ├── Footer.astro     # stopka + sitemap + dane firmy
│   │   ├── MobileMenu.tsx   # React island - burger menu
│   │   └── WhatsAppButton.tsx # React island - pływający button
│   ├── sections/
│   │   ├── Hero.astro       # sekcja hero z CTA
│   │   ├── Services.astro   # grid usług
│   │   ├── Trust.astro      # certyfikaty, liczby
│   │   ├── Testimonials.tsx # slider opinii (React)
│   │   ├── FAQ.astro        # accordion (CSS-only)
│   │   └── ContactSection.astro # adres + mapa + form
│   ├── seo/
│   │   ├── SEO.astro        # meta + OG + canonical
│   │   ├── SchemaLocalBusiness.astro
│   │   ├── SchemaService.astro
│   │   ├── SchemaPerson.astro
│   │   └── SchemaFAQ.astro
│   └── forms/
│       ├── ContactForm.tsx   # React island
│       ├── PriceCalculator.tsx # React island
│       └── BookingForm.tsx   # React island
├── content/
│   ├── services/             # MDX/MD opisy usług
│   ├── blog/                 # MDX artykuły bloga
│   └── opinions/             # Opinie klientów
├── layouts/
│   └── BaseLayout.astro      # wspólny layout
├── pages/
│   ├── index.astro
│   ├── kontakt.astro
│   ├── realizacje.astro
│   ├── wycena-kosztow-naprawy.astro
│   ├── wycena-wartosci-pojazdu.astro
│   ├── opis-stanu-technicznego.astro
│   ├── wycena-celno-skarbowa.astro
│   ├── doradztwo-zakupowe.astro
│   ├── pojazdy-zabytkowe-2.astro # ! zachowujemy URL
│   ├── zmiany-konstrukcyjne.astro
│   ├── pomoc-prawna.astro
│   ├── polisy-ubezpieczeniowe.astro
│   ├── biegly-skarbowy.astro
│   ├── biegly-sadowy.astro
│   ├── polityka-prywatnosci.astro
│   └── api/
│       └── contact.ts        # endpoint formularza
└── styles/
    └── global.css           # Tailwind base + custom
```

Migracja treści (drugi tydzień)

Treści **przepisujemy z poprawkami**, nie kopiujemy 1:1. Cel: dłuższe, bogatsze treści (+30-50% słów), z naturalnym keyword research.

Per strona:

- **Min. 600-800 słów** unikalnej treści
- **H1** z głównym keyword
- **H2/H3** ze wsparciem (related keywords)
- **Listy + tabele** (Google to lubi)
- **Internal links** do powiązanych usług (3-5 na stronie)
- **CTA** (przycisk kontaktu) na końcu + sticky
- **FAQ accordion** z 3-5 pytaniami na stronie usługi

SEO foundation per strona

Każda strona dostaje:

```
---
import BaseLayout from '../layouts/BaseLayout.astro';
import SchemaService from '../components/seo/SchemaService.astro';

const meta = {
  title: 'Wycena celno-skarbowa pojazdu - Rzeczoznawca certyfikowany | Motowycena',
  description: 'Profesjonalna wycena celno-skarbowa pojazdów sprowadzanych z zagranicy. Certyfikowany rzeczoznawca, akceptacja w US, szybki termin. Sprawdź ofertę.',
  ogImage: '/og/wycena-celno-skarbowa.jpg',
  canonical: 'https://www.motowycena.pl/wycena-celno-skarbowa/',
};
---
<BaseLayout meta={meta}>
  <SchemaService
    name="Wycena celno-skarbowa pojazdu"
    description="..."
    provider="Motowycena - Rafał Pelczar"
    areaServed="PL"
  />
  <!-- treść strony -->
</BaseLayout>
```

ETAP 2: Testowanie (Tydzień 3, dni 1-3)

Checklist pre-deploy

Lighthouse Performance ≥ 95 (każda strona)
Lighthouse SEO = 100 (każda strona)
Lighthouse Accessibility ≥ 95
Lighthouse Best Practices ≥ 95
Mobile-Friendly Test (Google)
Rich Results Test (schema markup waliduje się)
Cross-browser: Chrome, Firefox, Safari, Edge
Mobile: iPhone, Android (przynajmniej 2 urządzenia)
Wszystkie linki wewnętrzne działają (Screaming Frog)
Wszystkie obrazy ładują się + mają alt
Formularz kontaktowy → testowe wysłanie → mail dochodzi
Formularz – test spam (Cloudflare Turnstile blokuje boty)
WhatsApp button → otwiera czat
Sitemap.xml generuje się poprawnie
robots.txt allows all (po przeniesieniu z stagingu!)
Wszystkie 15 starych URL-i odpowiadają 200 OK
HTTPS bez błędów certyfikatu
Brak konsoli błędów JS w przeglądarce
Web Vitals real-user: CLS<0.1, LCP<2.5s, INP<200ms

Akceptacja klienta

Klient akceptuje:

- Treść każdej strony
- Wygląd (design)
- Dane kontaktowe (telefon, mail, adres)
- Logo, zdjęcia
- Politykę prywatności (aktualizacja pod RODO)

ETAP 3: Wdrożenie (Tydzień 3, dzień 4 - D-Day)

Plan godzinowy D-Day

Wykonujemy w nocy (np. 02:00-04:00) - wtedy ruch jest minimalny.

Godzina	Co	Kto	Czas
01:30	Finalny backup starej WP (pliki + DB)	Dev	15 min
01:45	Snapshot DNS records (zrzut ekranu)	Dev	5 min
02:00	DNS TTL -> 300s (z domyślnych 3600s)	Dev	5 min
02:05	Czekamy 60 min aż TTL spadnie globalnie	-	60 min

03:05	Deploy production build Astro do Cloudflare Pages	Dev	10 min
03:15	Test wszystkich 15 URL-i: 200 OK	Dev	10 min
03:25	Przełączenie DNS: A/CNAME -> Cloudflare Pages	Dev	5 min
03:30	DNS propagacja: test z różnych lokalizacji (dnschecker.org)	Dev	10 min
03:40	Wgrywam fallback .htaccess na starym hostingu z 301 wszystkim do nowej domeny - na wypadek gdyby DNS nie wyłączył starego serwera	Dev	10 min
03:50	Aktywuję Cloudflare WAF + DDoS protection	Dev	5 min
03:55	Usuwanie Basic Auth ze stagingu	Dev	2 min
03:57	Usuwanie meta noindex ze stagingu	Dev	3 min
04:00	Submit nowego sitemap do Google Search Console	Dev	5 min
04:05	URL Inspection top 5 stron -> "Request Indexing"	Dev	15 min
04:20	Submit do Bing Webmaster Tools	Dev	10 min
04:30	Smoke test: home + kontakt + 3 losowe usługi	Dev	15 min
04:45	Email do klienta: "wdrożone, sprawdź"	-	-

Procedura rollback (gdyby coś poszło źle)

1. DNS rollback: przełączenie A/CNAME spowrotem na stary hosting
→ propagacja w 5-10 min (bo TTL=300s)
2. Stara strona WP nadal istnieje (nic nie usuwaliśmy)
3. Klient ma mail "rollback wykonany, analizujemy problem"

ETAP 4: Post-mortem i monitoring (Tygodnie 4-12)

Tydzień 4 (D+1 do D+7)

Codzienna kontrola:

Rano:

- GSC → Coverage → Indexed pages (czy rosną?)
- GSC → Performance → Clicks/Impressions (vs baseline)
- GSC → Core Web Vitals → status
- Cloudflare Analytics → ruch ogólny
- UptimeRobot → uptime 100%

Wieczorem:

- Ręczna kontrola top 10 fraz w Google (czy strona widoczna)
- Sprawdzenie mailboxa biuro@motowycena.pl (czy formularze działają)

Tydzień 4-8 - stabilizacja

- **NORMA:** 10-20% spadek ruchu w pierwszych 2-3 tygodniach (Google przeindeksowuje)
- **OK:** wahania pozycji top 20 fraz o +/-5 pozycji
- **ALARM:** spadek >30% ruchu przez >2 tygodnie -> analiza

Tydzień 8-12 - beneficja

W tym okresie powinno być widać efekty:

- wzrost Impressions w GSC (więcej wyświetleń w wynikach)
- wzrost Average position (lepsze pozycje)
- wzrost CTR (dzięki meta descriptions)
- wzrost Pages per session (lepsza nawigacja)
- spadek Bounce rate (lepszy design)

Komunikacja z klientem

Co klient widzi

- **Tydzień 0:** prezentacja planu (ten dokument)
- **Tydzień 2:** pierwsza wersja na staging (do uwag)
- **Tydzień 3:** druga wersja na staging (do akceptacji)
- **Tydzień 3 D-Day:** deployment
- **Tydzień 4-8:** raport tygodniowy (PDF z metrykami)
- **Tydzień 12:** raport końcowy z porównaniem przed/po

Co klient ma robić

- Tydzień 0: dostarczyć dostępy + dane firmy
- Tydzień 1-2: dostarczyć treści/zdjęcia (jeśli chce nowe)
- Tydzień 2-3: akceptacja designu + treści
- Tydzień 4+: zbieranie opinii klientów (do osadzenia na stronie)

7. Checklist migracji

Pełna checklista migracji - odhaczaj jak idziesz

FAZA 0: Przed dotknięciem czegokolwiek

Dostępny

- ☐ Google Search Console - dodany jako "Owner" lub min. "Full user"
- ☐ Google Analytics - dostęp (jeśli klient ma)
- ☐ Hosting WordPress - FTP/SFTP/SSH lub cPanel
- ☐ Panel DNS - Cloudflare / OVH / domeny.tv / inny
- ☐ Skrzynka biuro@motowycena.pl - dostęp do odbioru testowych formularzy
- ☐ WordPress wp-admin - dostęp Admin

Backup

- ☐ Backup plików WP (cały folder public_html) -> archiwum lokalne
- ☐ Backup bazy danych WP -> plik .sql lokalnie
- ☐ Eksport plików media z `/wp-content/uploads/`
- ☐ Screenshoty wszystkich 15 podstron (Full Page, desktop + mobile)
- ☐ Zrzut konfiguracji DNS (wszystkie rekordy)

Inwentaryzacja

- ☐ Eksport URL-i z GSC: `Coverage -> Valid -> Export`
- ☐ Eksport top 100 fraz z GSC: `Performance -> Queries -> Export`
- ☐ Crawl Screaming Frog - pełny raport (15 URL-i)
- ☐ Test PageSpeed Insights na 5 reprezentatywnych podstronach (baseline)
- ☐ Test Mobile-Friendly na 5 podstronach
- ☐ Eksport backlinków (GSC -> Links -> External)

Setup nowego środowiska

- ☐ Konto Cloudflare (jeśli klient nie ma)
- ☐ Cloudflare Pages - nowy projekt
- ☐ Repository git (GitHub/GitLab)
- ☐ Subdomena `staging.motowycena.pl` skierowana na CF Pages
- ☐ Cloudflare Access - Basic Auth na staging
- ☐ `robots.txt` na staging: `User-agent: *\nDisallow: /`
- ☐ Meta noindex globalnie na staging

FAZA 1: Budowa (Tygodnie 1-2)

Setup projektu Astro

- `[] npm create astro@latest motowycena-new`
- `[] npx astro add react tailwind sitemap mdx`
- `[] npm install zod react-hook-form lucide-react`
- `[]` Konfiguracja `trailingSlash: 'always'`
- `[]` Konfiguracja `site: 'https://www.motowycena.pl'`
- `[]` TypeScript strict mode
- `[]` Setup shadcn/ui

SEO foundation

- `[]` Komponent `<SEO>` w `src/components/seo/SEO.astro`
- `[]` Komponent `<SchemaLocalBusiness>` (na każdej stronie)
- `[]` Komponent `<SchemaService>` (na stronach usług)
- `[]` Komponent `<SchemaPerson>` (rzeczoznawca)
- `[]` Komponent `<SchemaFAQ>` (na stronach z FAQ)
- `[]` Komponent `<Breadcrumbs>` + `<SchemaBreadcrumb>`
- `[]` Globalna konfiguracja `manifest.webmanifest` (PWA)
- `[]` Favicon + apple-touch-icon (wszystkie rozmiary)

Wszystkie 15 stron

- `[]` `/` (home)
- `[]` `/realizacje/`
- `[]` `/kontakt/`
- `[]` `/wycena-kosztow-naprawy/`
- `[]` `/wycena-wartosci-pojazdu/`
- `[]` `/opis-stanu-technicznego/`
- `[]` `/wycena-celno-skarbowa/`
- `[]` `/doradztwo-zakupowe/`
- `[]` `/polityka-prywatnosci/`
- `[]` `/pojazdy-zabytkowe-2/` (URL ZACHOWANY z `-2`)
- `[]` `/zmiany-konstrukcyjne/`
- `[]` `/pomoc-prawna/`
- `[]` `/polisy-ubezpieczeniowe/`
- `[]` `/biegly-skarbowy/`
- `[]` `/biegly-sadowy/`

Per strona checklist SEO

- `[]` Unikalny `<title>` (max 60 znaków, z keyword)
- `[]` Unikalny `<meta name="description">` (140-160 znaków, z CTA)

- ☐ OG image (1200x630px, JPG/PNG)
- ☐ OG title + description
- ☐ Twitter card (`summary_large_image`)
- ☐ Canonical link (self-referencing)
- ☐ H1 z głównym keyword
- ☐ H2/H3 z related keywords
- ☐ Min. 600 słów treści
- ☐ Min. 3 internal links
- ☐ Wszystkie obrazy z `alt`
- ☐ Wszystkie obrazy w WebP/AVIF + fallback
- ☐ Lazy loading na obrazach poniżej viewport
- ☐ Schema markup (Service/LocalBusiness)
- ☐ FAQ accordion (gdzie pasuje) + Schema FAQPage
- ☐ CTA na końcu strony

Komponenty interaktywne (React islands)

- ☐ Formularz kontaktowy z walidacją zod
- ☐ Mobile menu (burger)
- ☐ Pływający WhatsApp button
- ☐ Cookie banner (RODO)
- ☐ Slider opinii klientów
- ☐ Kalkulator wstępnej wyceny

Backend / API

- ☐ Endpoint `POST /api/contact` (Astro endpoint)
- ☐ Integracja Resend (lub SMTP klienta)
- ☐ Cloudflare Turnstile (anti-spam)
- ☐ Rate limiting (5 req/min/IP)
- ☐ Honeypot field (anti-bot)
- ☐ Walidacja serwerowa (zod)
- ☐ Log każdego zapytania (do CSV/DB)

FAZA 2: Pre-deploy testy (Tydzień 3)

Performance

- ☐ Lighthouse Performance ≥ 95 na każdej stronie
- ☐ LCP $< 2.5s$ na 3G
- ☐ CLS < 0.1
- ☐ INP $< 200ms$
- ☐ First Contentful Paint $< 1.5s$

- ☐ Total bundle JS < 100 KB (gzip) na większości stron
- ☐ Brak render-blocking scripts

SEO

- ☐ Lighthouse SEO = 100 na każdej stronie
- ☐ Wszystkie meta description unikalne
- ☐ Wszystkie title tags unikalne
- ☐ Każda strona ma jeden i tylko jeden H1
- ☐ Schema validuje się w Rich Results Test
- ☐ Sitemap.xml generuje się i listuje wszystkie strony
- ☐ robots.txt na produkcję (`Allow: /` , sitemap)

Dostępność

- ☐ Lighthouse Accessibility >= 95
- ☐ Kontrast tekstu >= 4.5:1
- ☐ Wszystkie obrazy z alt
- ☐ Formularze z label
- ☐ Nawigacja klawiaturą działa
- ☐ Focus states widoczne

Cross-browser

- ☐ Chrome (desktop + mobile)
- ☐ Firefox
- ☐ Safari (macOS + iOS)
- ☐ Edge
- ☐ Samsung Internet (Android)

Funkcjonalność

- ☐ Formularz wysyła testowy mail
- ☐ WhatsApp button otwiera czat
- ☐ Telefon (`tel:`) działa na mobile
- ☐ Mail (`mailto:`) działa
- ☐ Wszystkie linki wewnętrzne 200 OK
- ☐ Brak linków na 404
- ☐ Hreflang (jeśli używamy multilang)

FAZA 3: D-Day deployment

Pre-deploy (15 min)

- ☐ Finalny backup WP

- ☐ DNS TTL -> 300s
- ☐ Klient powiadomiony o oknie wdrożenia

Deploy (30 min)

- ☐ Production build Astro: `astro build`
- ☐ Deploy na Cloudflare Pages (production environment)
- ☐ Test wszystkich 15 URL-i przez Cloudflare Pages URL
- ☐ Status każdego URL: 200 OK
- ☐ Test schema na Rich Results Test (3 reprezentatywne strony)

DNS switch (10 min)

- ☐ Backup DNS records (kopia tekstowa)
- ☐ Zmiana `A` record `motowycena.pl` -> CF Pages
- ☐ Zmiana `CNAME` `www.motowycena.pl` -> CF Pages
- ☐ Test propagacji DNS (dnschecker.org)
- ☐ Test z 5 lokalizacji geograficznych

Post-deploy (45 min)

- ☐ Cloudflare WAF aktywny
- ☐ HTTPS certyfikat aktywny (CF Universal SSL)
- ☐ HSTS włączony
- ☐ Cache rules ustawione (static assets na 1 rok)
- ☐ Robots.txt usuwa `Disallow: /`
- ☐ Meta noindex usunięte ze wszystkich stron
- ☐ Submit sitemap do GSC
- ☐ URL Inspection top 10 stron -> "Request Indexing"
- ☐ Bing Webmaster Tools - submit sitemap
- ☐ Email do klienta: "wdrożone"

FAZA 4: Monitoring post-deploy

Codziennie (pierwszy tydzień)

- ☐ GSC Coverage: czy nowe strony są indexed
- ☐ GSC Performance: clicks + impressions trend
- ☐ Uptime (UptimeRobot, BetterStack)
- ☐ Cloudflare Analytics: ruch ogólny
- ☐ Email box: testowe formularze działają
- ☐ Top 5 fraz: pozycja w Google (ręcznie)

Co tydzień (tygodnie 2-12)

- ☐ Raport pozycji top 20 fraz
- ☐ Raport ruchu organic
- ☐ Raport konwersji (form submissions)
- ☐ Core Web Vitals (GSC + PageSpeed Insights)
- ☐ Crawl errors w GSC

Po 30 dniach

- ☐ Pełny crawl Screaming Frog (porównanie z baseline)
- ☐ Audyt linków przychodzących (broken backlinks?)
- ☐ Optymalizacja stron które mają najgorszy CTR
- ☐ Update treści które tracą pozycję

Po 90 dniach

- ☐ Pełny raport ROI dla klienta
- ☐ Porównanie metryk: przed/po
- ☐ Plan rozwoju (blog, content marketing, link building)

8. Konkretnie ulepszenia SEO

Konkretnie ulepszenia SEO - co dokładnie wdrożymy

Ten dokument opisuje **każde** ulepszenie SEO które wdrożymy na nowej stronie. Każde z nich ma uzasadnienie biznesowe i mierzalny efekt.

1. Meta tagi - 100% pokrycia

Obecnie: 0/15 stron ma meta description

Po migracji: 15/15 + na każdej nowej stronie

Format meta title:

[Keyword główny] - [USP] | Motowycena

Przykłady:

- Wycena celno-skarbowa pojazdu - Certyfikowany rzeczoznawca | Motowycena
- Wycena wartości pojazdu - Profesjonalna ekspertyza w 24h | Motowycena
- Rzeczoznawca samochodowy Wielkopolska - Rafał Pelczar | Motowycena

Reguły:

- Max 60 znaków
- Główny keyword na początku
- Marka na końcu
- Modyfikator (szybko, profesjonalnie, certyfikowany) w środku

Format meta description:

[Co robisz]. [Wyróżnik/zalety]. [Region/zasięg]. [CTA].

Przykład dla </wycena-celno-skarbowa/> :

Profesjonalna wycena celno-skarbowa pojazdów sprowadzanych z zagranicy.
Certyfikowany rzeczoznawca (RS001771), akceptacja w urzędach skarbowych,
termin 24-48h. Sprawdź ofertę i zamów online.

Reguły:

- 140-160 znaków
- Główny + secondary keyword
- Co najmniej 1 modyfikator zaufania (numer certyfikatu, lata doświadczenia)
- CTA ("Sprawdź", "Zamów", "Zadzwoń")

2. Schema.org markup - pełny pakiet

LocalBusiness (na każdej stronie, w `<head>`)

```
{
  "@context": "https://schema.org",
  "@type": "LocalBusiness",
  "@id": "https://www.motowycena.pl/#business",
  "name": "Motowycena - Rzecznawca Techniki Samochodowej Rafał Pelczar",
  "image": "https://www.motowycena.pl/logo.jpg",
  "telephone": "+48509146666",
  "email": "biuro@motowycena.pl",
  "url": "https://www.motowycena.pl",
  "priceRange": "$$",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "ul. Spacerowa 10",
    "addressLocality": "Garki",
    "postalCode": "63-430",
    "addressCountry": "PL"
  },
  "geo": {
    "@type": "GeoCoordinates",
    "latitude": 51.6,
    "longitude": 17.7
  },
  "areaServed": [
    {"@type": "AdministrativeArea", "name": "Wielkopolska"},
    {"@type": "AdministrativeArea", "name": "Dolnośląskie"},
    {"@type": "City", "name": "Poznań"},
    {"@type": "City", "name": "Wrocław"},
    {"@type": "City", "name": "Warszawa"},
    {"@type": "City", "name": "Kraków"}
  ],
  "openingHoursSpecification": {
    "@type": "OpeningHoursSpecification",
    "dayOfWeek": ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"],
    "opens": "08:00",
    "closes": "18:00"
  },
  "founder": {
    "@type": "Person",
    "name": "Rafał Pelczar",
    "jobTitle": "Rzecznawca Techniki Samochodowej",
    "hasCredential": "Certyfikat nr RS001771"
  }
}
```

Service (na każdej stronie usługi)

```
{
  "@context": "https://schema.org",
  "@type": "Service",
  "serviceType": "Wycena celno-skarbowa pojazdu",
  "provider": {"@id": "https://www.motowycena.pl/#business"},
  "areaServed": "PL",
  "offers": {
    "@type": "Offer",
    "priceCurrency": "PLN",
    "priceRange": "200-500 PLN"
  }
}
```

FAQPage (gdzie wstawimy FAQ)

```
{
  "@context": "https://schema.org",
  "@type": "FAQPage",
  "mainEntity": [{
    "@type": "Question",
    "name": "Ile kosztuje wycena celno-skarbowa?",
    "acceptedAnswer": {
      "@type": "Answer",
      "text": "Koszt wyceny celno-skarbowej zależy od..."
    }
  }]
}
```

BreadcrumbList (na każdej podstronie)

```
{
  "@context": "https://schema.org",
  "@type": "BreadcrumbList",
  "itemListElement": [
    {"@type": "ListItem", "position": 1, "name": "Strona główna", "item": "https://www.motowycena.pl/"},
    {"@type": "ListItem", "position": 2, "name": "Usługi", "item": "https://www.motowycena.pl/uslugi/"},
    {"@type": "ListItem", "position": 3, "name": "Wycena celno-skarbowa", "item": "https://www.motowycena.pl/wycena-celno-skarbowa/"}
  ]
}
```


Person (na stronie /o-mnie/)

```
{
  "@context": "https://schema.org",
  "@type": "Person",
  "name": "Rafał Pelczar",
  "jobTitle": "Rzecznik Techniki Samochodowej",
  "telephone": "+48509146666",
  "email": "biuro@motowycena.pl",
  "hasCredential": [{
    "@type": "EducationalOccupationalCredential",
    "credentialCategory": "certification",
    "identifier": "RS001771",
    "name": "Certyfikat Rzecznik Techniki Samochodowej"
  }],
  "worksFor": { "@id": "https://www.motowycena.pl/#business" }
}
```

3. Open Graph + Twitter Cards

Każda strona dostaje OG tagi:

```
<meta property="og:type" content="website" />
<meta property="og:site_name" content="Motowycena" />
<meta property="og:title" content="..." />
<meta property="og:description" content="..." />
<meta property="og:url" content="https://www.motowycena.pl/..." />
<meta property="og:image" content="https://www.motowycena.pl/og/..." />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
<meta property="og:locale" content="pl_PL" />

<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="..." />
<meta name="twitter:description" content="..." />
<meta name="twitter:image" content="https://www.motowycena.pl/og/..." />
```

OG images generowane automatycznie:

- 1200x630px
- Tło z logiem
- Tytuł strony jako tekst
- Branding (RS001771)
- Format: JPG (mniejszy niż PNG)

4. Core Web Vitals - cel: zielony status w GSC

Metryka	Cel	Jak osiągnąć
LCP (Largest Contentful Paint)	<2.5s	Astro generuje statyczny HTML, obrazy WebP + lazy loading, fonty self-hosted
INP (Interaction to Next Paint)	<200ms	Minimum JS, React tylko w islands

CLS (Cumulative Layout Shift)	<0.1	Width/height na każdym obrazie, font-display: swap, brak skoków
-------------------------------	------	--

5. Wewnętrzne linkowanie

Obecnie: prawdopodobnie tylko menu główne

Po migracji: bogata sieć linków

Każda strona usługi linkuje do:

- 2-3 powiązanych usług ("Może Cię również zainteresować")
- Strony kontaktu (CTA w treści + footer)
- Bloga (gdy będzie)
- FAQ (gdy będzie)

Internal links flow:

Home → wszystkie usługi (1 click)
 Usługa → powiązane usługi + FAQ + kontakt + opinie
 Blog post → 2-3 usługi (link contextual)
 FAQ → konkretna usługa odpowiadająca pytaniu

6. Hreflang (na przyszłość)

Jeśli klient zechce wersję angielską/niemiecką dla emigrantów wracających do PL:

```
<link rel="alternate" hreflang="pl" href="https://www.motowycena.pl/" />
<link rel="alternate" hreflang="en" href="https://www.motowycena.pl/en/" />
<link rel="alternate" hreflang="de" href="https://www.motowycena.pl/de/" />
<link rel="alternate" hreflang="x-default" href="https://www.motowycena.pl/" />
```

Astro wspiera to natywnie ([@astrojs/i18n](#)).

7. Treść - ekspansja o 30-50%

Obecne strony mają w okolicach 200-400 słów. Nowa wersja:

- **Min. 600 słów** każda strona usługi
- **Naturalne wystąpienia keyword** (1-2% density)
- **Powiązane słowa kluczowe (LSI)** - synonimy, wariacje
- **Listy + tabele** (Google to lubi)
- **Cytaty i case studies** (jeśli klient ma)
- **Sekcja "Jak to działa"** krok po kroku
- **Sekcja "Dla kogo"** (segmentacja klientów)
- **Sekcja "Co zawiera wycena"** (transparentność)
- **Sekcja FAQ** (3-5 pytań na stronie)

8. Świeżość treści (Fresh content signal)

Google premiuje świeżą treść. Strategie:

- **Daty** `lastmod` w **sitemapie** - aktualizowane przy każdej edycji
- `<time datetime="...">` widoczny na blogu
- **Blog z 2-4 artykułami miesięcznie** (długoterminowy plan)
- **Aktualizacje przy zmianach prawnych** (np. nowe przepisy o wycenach celno-skarbowych)

9. Sitemap XML - rozbudowany

Astro `@astrojs/sitemap` generuje sitemap automatycznie. Możemy dodać:

- **priority** per page (home = 1.0, usługi = 0.9, blog = 0.7)
- **changefreq** per typ strony
- **lastmod** z gita (commit date)
- **Subsitemapy** osobne dla: pages, blog, images
- **Image sitemap** (osobny sitemap dla zdjęć)

10. Local SEO - dla emigrantów i kierowców z całej Polski

Podstrony lokalne (P2 w roadmapie)

```
/rzeczoznawca-poznan/  
/rzeczoznawca-wroclaw/  
/rzeczoznawca-kalisz/  
/rzeczoznawca-warszawa/  
/rzeczoznawca-krakow/  
/rzeczoznawca-lodz/  
/rzeczoznawca-bydgoszcz/  
/rzeczoznawca-gdansk/
```

Każda taka strona:

- Title: `Rzeczoznawca samochodowy [Miasto] - Wyceny pojazdów | Motowycena`
- Treść 500+ słów o specyfice miasta (np. komis, autoryzowane stacje)
- Lista usług + ceny
- Mapa Google z punktem (siedziba lub punkt dojazdu)
- Schema `LocalBusiness` z `areaServed: [Miasto]`
- Linki do home + powiązane usługi

Google Business Profile

- ☐ Setup/optimalizacja GBP
- ☐ Min. 10 zdjęć (rzeczoznawca w pracy, dokumentacja, siedziba)
- ☐ Wszystkie kategorie biznesu wypełnione
- ☐ Godziny otwarcia
- ☐ Posty 2x w miesiącu
- ☐ Aktywne odpowiadanie na opinie

11. Performance budget

Każda strona musi spełniać:

Zasób	Limit
HTML	<30 KB
CSS	<50 KB
JavaScript (krytyczny)	<20 KB
JavaScript (lazy)	<80 KB
Obrazy (above-fold)	<200 KB total
Fonty	<50 KB (2 wagi, woff2)
Total wagę strony	<400 KB
Time to First Byte	<600ms

12. Mobile-first design

- Wszystkie elementy testowane na mobile FIRST
- Touch targets $\geq 48 \times 48$ px
- Brak hover-only interactions
- Sticky CTA (Zadzwoń / WhatsApp) na mobile
- Responsywne obrazy (`<picture>` z `srcset`)

13. RODO compliance (bonus SEO)

Google premiuje strony privacy-friendly:

- Cookie banner z prawdziwym wyborem (nie dark pattern)
- Brak Google Fonts (self-hosted) - mniej zewnętrznych żądań
- Brak Google Analytics (lub min. anonimizacja IP + cookieless mode)
- Cloudflare Web Analytics zamiast GA
- Polityka prywatności aktualna pod RODO + Akt o usługach cyfrowych (DSA)

14. Monitoring i pomiary

Po wdrożeniu codziennie sprawdzamy:

- **GSC**: Performance, Coverage, CWV, Mobile Usability, Rich Results
- **PageSpeed Insights**: per-page check
- **Cloudflare Analytics**: ruch, bot traffic, geo
- **UptimeRobot**: uptime 99.9%+
- **Sentry / LogRocket** (opcjonalnie): błędy JS w przeglądarce

Spodziewane wyniki po 3 miesiącach

Metryka	Przed	Po (cel)	Wzrost
Pages indexed	15	25-30 (+ blog)	+60-100%
Lighthouse Performance	~60	95+	+58%
Average position w Google	~25	12-15	poprawa 10 pozycji
Impressions / mc	baseline	+200%	2-3x
Clicks / mc	baseline	+150%	2.5x
CTR	~2%	4-5%	2x (meta descriptions!)
Rich snippets	0	5-10	+∞
Mobile usability errors	nieznane	0	-
Core Web Vitals (zielone)	nieznane	100%	-

Spodziewane wyniki po 12 miesiącach (z blogiem)

- **+500% impressions** (baseline -> 5x)
- **+300% clicks** (baseline -> 4x)
- **TOP 3 pozycje** na main keywords: "rzeczoznawca samochodowy Wielkopolska", "wycena celno-skarbowa", "biegły sądowy techniki samochodowej"
- **DA/DR wzrost** (z budowaniem linków)
- **Brand search +200%** (więcej osób szuka "Motowycena Pelczar")

CZESC IV

Setup techniczny

9. Turnstile + Resend - integracja

Cloudflare Turnstile + Resend - setup

Endpoint `/api/contact` ma już całą logikę. Wystarczy uzupełnić zmienne środowiskowe.

1. Resend (wysyłka maili)

Setup

- 1 Załóż konto: <https://resend.com> (darmowe 3000 maili/mc, 100/dzień)
- 2 Dashboard -> API Keys -> Create API Key (uprawnienia: Sending access)
- 3 Skopiuj klucz `re_XXXXXXXXXXXX`
- 4 Domena - dwie opcje:
 - **Testowo:** użyj domeny `resend.dev` (gotowa do testów, bez konfiguracji DNS)
 - **Produkcyjnie:** Dashboard -> Domains -> Add Domain -> `motowycena.pl`, ustaw rekordy DNS (SPF, DKIM)

Zmienne (`.env` lokalnie, Cloudflare Pages na produkcji)

```
RESEND_API_KEY=re_XXXXXXXXXXXX
CONTACT_EMAIL=biuro@motowycena.pl
MAIL_FROM=Motowycena <noreply@motowycena.pl> # po weryfikacji domeny
# Lub na testy:
# MAIL_FROM=onboarding@resend.dev
```

Bez Resend (tryb dev)

Endpoint **nie wywali błędu** - po prostu zaloguje payload do konsoli zamiast wysłać. Idealne do testów lokalnych.

2. Cloudflare Turnstile (anti-spam)

Dlaczego nie reCAPTCHA?

- Darmowy, brak limitów
- Bez Google (RODO-friendly)
- Niewidzialny dla większości użytkowników
- Mniejszy bundle JS (~40KB vs ~80KB w reCAPTCHA)

Setup

- 1 <https://dash.cloudflare.com> -> Turnstile -> Add site
- 2 Widget name: `motowycena.pl`
- 3 Hostname: `motowycena.pl` (i `staging.motowycena.pl` jeśli chcesz)
- 4 Widget mode: **Managed** (automatyczna detekcja botów)
- 5 Skopiuj:

- **Site Key** (publiczny) -> `PUBLIC_TURNSTILE_SITE_KEY`
- **Secret Key** (server-side) -> `TURNSTILE_SECRET_KEY`

Zmienne

```
PUBLIC_TURNSTILE_SITE_KEY=0x4AAAAAAxxxxxxxxxxxxx
TURNSTILE_SECRET_KEY=0x4AAAAAAxxxxxxxxxxxxx
```

Jak to działa

- Widget pojawia się w formularzu (jeśli `PUBLIC_TURNSTILE_SITE_KEY` jest ustawione)
- Po submit, frontend wysyła token w polu `cf-turnstile-response`
- Endpoint weryfikuje token z Cloudflare (`/turnstile/v0/siteverify`)
- Jeśli weryfikacja nieudana -> odrzucamy zapytanie

Bez Turnstile

Jeśli zmienne nie są ustawione - endpoint **pomija weryfikację** (tryb dev). Honeypot dalej działa jako podstawowa ochrona.

3. Cloudflare Pages - Environment Variables

Dashboard -> Pages -> motowycena -> Settings -> Environment Variables:

Variable	Type	Value
<code>RESEND_API_KEY</code>	Secret	<code>re_XXXXX</code>
<code>CONTACT_EMAIL</code>	Plain	<code>biuro@motowycena.pl</code>
<code>MAIL_FROM</code>	Plain	<code>Motowycena <noreply@motowycena.pl></code>
<code>PUBLIC_TURNSTILE_SITE_KEY</code>	Plain	<code>0x4AAAAA...</code>
<code>TURNSTILE_SECRET_KEY</code>	Secret	<code>0x4AAAAA...</code>
<code>PUBLIC_CLOUDFLARE_ANALYTICS_TOKEN</code>	Plain	<code>tokenz cf analytics</code>

Wszystkie zmienne z `PUBLIC_` prefiksem są bundlowane do frontu - nie wrzucaj tam sekretów.

4. Test po setupie

```
# Lokalnie
cd app
cp .env.example .env
# uzupełnij wartości w .env
npm run dev
# otwórz http://localhost:4321/kontakt/
# wyślij testowy formularz - mail powinien dojść na CONTACT_EMAIL
```

5. Plan B - SMTP klienta

Jeśli klient woli wysyłać przez własny SMTP (np. `m.biuro@motowycena.pl` ze swojego hostingu):


```
// W endpoint contact.ts zamiast Resend API:
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  host: 'smtp.klient.pl',
  port: 587,
  secure: false,
  auth: {
    user: import.meta.env.SMTP_USER,
    pass: import.meta.env.SMTP_PASS,
  },
});

await transporter.sendMail({
  from: FROM,
  to: CONTACT_EMAIL,
  replyTo: data.email,
  subject,
  html,
});
```

Wadą tej opcji: nodemailer to ~500 KB więcej w bundle (vs ~5 KB dla fetch do Resend), więc Resend jest lepszym wyborem dla statycznej strony na Cloudflare Pages.

CZESC V

Encyklopedia techniczna

Dla developera: każde pojecie z 4 warstwami - po ludzku, definicja techniczna, mechanika pod spodem, kod z naszego projektu. Czytaj zeby w *chuj* sie doszkolic.

10. Pojecia + definicje + kod

Encyklopedia techniczna projektu

Dla każdej rzeczy używanej w tym projekcie:

- 1 **Po ludzku** - jedno zdanie dla klienta
- 2 **Definicja techniczna** - nazewnictwo branżowe
- 3 **Mechanika** - jak to działa pod spodem
- 4 **Kod** - realny przykład z naszego projektu

A. FRAMEWORK I FRONTEND

A.1. Astro 5

Po ludzku: Astro buduje strony jak gazety - cała treść drukowana na papierze w drukarni, gotowa do czytania. Zero ładowania w głowie czytelnika.

Definicja techniczna: Astro to **content-first meta-framework** używający paradygmatu **MPA (Multi-Page Application)** z **Islands Architecture**. Generuje statyczny HTML w build-time (SSG - Static Site Generation), z opcjonalnymi "wyspami" interaktywności dla wybranych komponentów.

Mechanika pod spodem:

- 1 `astro build` parsuje pliki `.astro`, `.mdx`, `.ts` z `src/pages/`
- 2 Każdy `.astro` to gotowy HTML + (opcjonalnie) hydrowane wyspy
- 3 Komponenty React/Vue/Svelte używane bez dyrektywy `client:*` renderują się **tylko serwerowo** - zerowy JS w przeglądarce
- 4 Komponenty z `client:load` / `client:idle` / `client:visible` dostają oddzielne chunki JS ładowane warunkowo

Kod z projektu:

```
---
// src/pages/index.astro - wszystko poniżej tej linii to PURE HTML w build time
import Hero from '@components/sections/Hero.astro'; // → server only, 0 KB JS
import ContactForm from '@components/forms/ContactForm.tsx'; // React island
---
<html>
  <body>
    <Hero />                                <!-- statyczny HTML -->
    <ContactForm client:visible />           <!-- React hydruje gdy widoczne -->
  </body>
</html>
```

Konkurencja: Next.js (heavier, app-like), Remix (data-driven SSR), 11ty (lighter, no JS framework).

A.2. Islands Architecture

Po ludzku: Strona to gazeta z 1-2 interaktywnymi miejscami (formularz, mapa). Reszta to statyczny tekst. Tylko te interaktywne miejsca ładują JavaScript.

Definicja techniczna: Architecture pattern (Jason Miller, 2020) gdzie strona to statyczny HTML z **odizolowanymi wyspami hydratacji**. Każda wyspa to niezależny komponent z własnym chunkiem JS, własną hydratacją, niekomunikujący się z innymi wyspami via JS tree.

Mechanika:

- Build-time: każdy komponent z `client:*` to osobny bundle `_astro/ContactForm.HASH.js`
- Runtime: Astro wstrzykuje runtime ~3 KB (`@astrojs/runtime`) który orkestruje lazy hydration
- Strategie hydratacji:
- `client:load` -> hydrate on page load
- `client:idle` -> `requestIdleCallback()` - po idle main thread
- `client:visible` -> `IntersectionObserver` - gdy w viewporcie
- `client:media="(max-width: 768px)"` -> tylko gdy matchuje media query
- `client:only="react"` -> zero SSR, tylko client (gdy komponent używa np. `window`)

Kod z projektu:

```
<!-- src/layouts/BaseLayout.astro -->
<WhatsAppButton client:idle phone={business.contact.whatsapp} />
<!--           ^^^^^^^^^ Hydratacja w bezczynnym czasie main threada
                    Wyspa nie blokuje LCP -->

<!-- src/pages/kontakt.astro -->
<ContactForm client:visible turnstileSiteKey={turnstileSiteKey} />
<!--           ^^^^^^^^^^^^^ IntersectionObserver - nie hydruje dopóki klient
                    nie przewinie do formularza -->
```

A.3. SSG vs SSR vs ISR vs CSR

Po ludzku:

- **SSG:** gazeta - wydrukowana raz, daje się każdemu klientowi
- **SSR:** gazeta drukowana na życzenie - każdy klient czeka aż się wydrukuje
- **ISR:** gazeta wydrukowana z opcją "odśwież co 5 minut"
- **CSR:** klient dostaje pustą kartkę i pisze sobie sam (czeka długo)

Definicje techniczne:

Skrót	Pełna nazwa	Kiedy generowany HTML
SSG	Static Site Generation	Build time, raz
SSR	Server-Side Rendering	Per request, na serwerze
ISR	Incremental Static Regeneration	Build + revalidate co N sekund
CSR	Client-Side Rendering	W przeglądarce po pobraniu pustego HTML

PPR	Partial Prerendering (Next.js 16)	SSG dla statycznych części + SSR dla dynamic holes
-----	-----------------------------------	--

Kod z projektu:

```
// astro.config.mjs - domyślnie SSG
export default defineConfig({
  output: 'static', // wszystkie strony pre-renderowane w build
});

// Per-strona można włączyć SSR:
// src/pages/api/contact.ts
export const prerender = false; // ta strona nie SSG-uje się, SSR per request
```

A.4. React 19 jako wyspa

Po ludzku: Używamy React tylko tam gdzie jest realna interaktywność - formularz, slider, mapa. Reszta strony to "głupi" HTML.

Definicja techniczna: React 19 wprowadza **Server Components (RSC)** ale w Astro **nie używamy RSC** - używamy klasycznych client components renderowanych przez Astro server-side raz, hydrowanych w przeglądarce. Nowości React 19 które wykorzystujemy: `use()` hook, ulepszony `useFormStatus`, `useOptimistic`, automatyczny React Compiler.

Mechanika hydration w Astro:

- 1 Astro renderuje React komponent po stronie serwera (przez `react-dom/server`)
- 2 Wynikowy HTML trafia do strony
- 3 JS bundle z komponentem łąduje w `<script type="module">`
- 4 Po spełnieniu dyrektywy `client:*` -> `hydrateRoot(element, <Component {...props} />)`

Kod z projektu:

```
// src/components/forms/ContactForm.tsx
import { useState, useRef, type FormEvent } from 'react';
import { z } from 'zod';

// Schema walidacji - runtime check (zod nie znika po kompilacji TS)
const schema = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  // ...
});

export default function ContactForm({ turnstileSiteKey }: Props) {
  const [status, setStatus] = useState<Status>('idle');
  // useState to React hook - wymaga client component → potrzebujemy client:*
}

<!-- W .astro: -->
<ContactForm client:visible turnstileSiteKey={import.meta.env.PUBLIC_TURNSTILE_SITE_KEY} />
```

A.5. TypeScript strict mode

Po ludzku: Komputer pyta "czy na pewno?" przy każdej operacji. Mniej bugów, więcej wstrząsania głową.

Definicja techniczna: Flagi w `tsconfig.json` które wymuszają najściślejsze sprawdzanie typów: `strict: true` aktywuje `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, `noImplicitThis`, `useUnknownInCatchVariables`, `alwaysStrict`.

Co to wymusza:

- `null` i `undefined` muszą być explicit - `Foo | null` zamiast nadmiarowych `?.`
- Każdy parametr funkcji ma typ - żadnych `any` ukrytych
- `catch (e)` daje `e: unknown` zamiast `e: any` - wymusza `instanceof Error` check
- Class properties muszą być inicjowane w konstruktorze lub `!`

Kod z projektu:

```
// app/tsconfig.json
{
  "extends": "astro/tsconfigs/strict",
  "compilerOptions": {
    "paths": {
      "@components/*": ["src/components/*"]
    }
  }
}

// app/src/pages/api/contact.ts
async function verifyTurnstile(
  token: string,
  secret: string,
  ip: string | undefined, // explicit "może być undefined"
): Promise<boolean> {
  try {
    // ...
  } catch {
    return false; // strict: catch bez (e) bo i tak nieużywane
  }
}
```

A.6. Tailwind CSS 4

Po ludzku: Zamiast pisać własny CSS dla każdej rzeczy, składamy klasy z gotowych klocków (`bg-blue-500 p-4 rounded-lg`).

Definicja techniczna: Utility-first CSS framework. Klasy są atomic (jedna klasa = jedna własność CSS). Build używa **JIT (Just-In-Time)** compiler - generuje TYLKO klasy faktycznie użyte w HTML/JSX. Wynikowy CSS bundle bywa 5-20 KB zamiast typowych 100 KB+.

Mechanika v4:

- Tailwind v4 używa **CSS Cascade Layers** (`@layer base/components/utilities`)
- Custom theme tokens definiuje się w CSS variable
- Brak `tailwind.config.js` - konfiguracja w CSS

Kod z projektu:

```

/* app/src/styles/global.css */
@import "tailwindcss";

@theme {
  --color-brand-500: #1e3a8a;
  --color-brand-600: #1e40af;
  --font-sans: 'Inter', system-ui, sans-serif;
}

<!-- Użycie -->
<button class="bg-brand-600 hover:bg-brand-700 text-white px-6 py-3 rounded-lg">
  Zadzwoń
</button>

```

Generowany CSS (output):

```

.bg-brand-600 { background-color: var(--color-brand-600); }
.hover\:bg-brand-700:hover { background-color: var(--color-brand-700); }
.text-white { color: #fff; }
.px-6 { padding-left: 1.5rem; padding-right: 1.5rem; }
.py-3 { padding-top: 0.75rem; padding-bottom: 0.75rem; }
.rounded-lg { border-radius: 0.5rem; }

```

A.7. Content Collections + Zod schema

Po ludzku: Folder z plikami treści gdzie komputer pilnuje że każdy plik ma wszystkie wymagane pola (tytuł, opis, data).

Definicja techniczna: Astro feature dla **typed content management**. Pliki MDX/MD w `src/content/{collection}/` walidowane przez zod schema przy każdym buildzie. Daje **end-to-end type safety** od pliku tekstowego do React props.

Mechanika:

- 1 `defineCollection()` deklaruje strukturę
- 2 Astro generuje typy TypeScript z schemy (`astro:content`)
- 3 Build-time: każdy plik MDX musi pasować do schemy lub `astro:build:setup` zwraca error
- 4 `getCollection('services')` zwraca w pełni typowaną tablicę

Kod z projektu:

```

// app/src/content/config.ts
import { defineCollection, z } from 'astro:content';

const services = defineCollection({
  type: 'content',
  schema: ({ image }) => z.object({
    title: z.string().min(3).max(80),
    seo: z.object({
      title: z.string().min(10).max(70),
      description: z.string().min(50).max(170),
      keywords: z.array(z.string()).default([]),
    }),
    benefits: z.array(z.string()).min(2).max(8),
    priceRange: z.object({
      from: z.number().int().positive(),
      to: z.number().int().positive(),
      currency: z.literal('PLN').default('PLN'),
    })
  })
});

```

```

    }).optional(),
    publishedAt: z.coerce.date().optional(),
  }),
});

export const collections = { services };

---
title: Wycena celno-skarbowa
seo:
  title: Wycena celno-skarbowa pojazdu - Rzeczoznawca
  description: Profesjonalna wycena celno-skarbowa pojazdów z zagranicy...
  keywords: [wycena celno-skarbowa, akcyza]
benefits:
  - Akceptowane w UCS w całej Polsce
  - Termin 24-48h
priceRange:
  from: 300
  to: 600
---

## Czym jest wycena celno-skarbowa?

Treść MDX (Markdown + komponenty React inline)...

// W stronie - fully typed:
import { getCollection } from 'astro:content';
const services = await getCollection('services');
//      ^? CollectionEntry<'services'>[]
services[0].data.benefits; // string[] - TS wie że to string array

```

B. SEO TECHNICZNE

B.1. Meta description

Po ludzku: Krótki podpis pod tytułem w Google - reklama strony przed kliknięciem.

Definicja techniczna: HTML meta tag `<meta name="description" content="...">` w `<head>`.

Nie jest czynnikiem rankingowym, ALE wpływa na CTR (Click-Through Rate) z SERP-a. Google używa jako snippet gdy treść matchuje zapytanie; w przeciwnym razie generuje własny z body content.

Reguły:

- Długość: 140-160 znaków (powyżej Google ucina `...`)
- Język matchujący `<html lang="pl">`
- Słowa kluczowe naturalnie umieszczone (nie keyword stuffing)
- Powinno zawierać CTA (call-to-action)

Kod z projektu:

```

<!-- app/src/components/seo/SEO.astro -->
<meta name="description" content={description} />

<!-- Użycie w stronie: -->
<BaseLayout
  title="Wycena celno-skarbowa pojazdu - Rzeczoznawca certyfikowany"

```



```
description="Profesjonalna wycena celno-skarbowa pojazdów z zagranicy. Akceptowana w urzędac  
h celno-skarbowych. Certyfikowany rzeczoznawca (RS001771), termin 24-48h."  
</>
```

B.2. Schema.org JSON-LD

Po ludzku: Strukturalna wizytówka dla Google - mówi "tu jest firma, telefon, adres, godziny". Google wyświetla bogato (gwiazdki, mapy, godziny w SERP).

Definicja techniczna: Structured data w formacie **JSON-LD (JSON for Linking Data)**. Standardy schema.org (consortium Google + Yahoo + Microsoft + Yandex). Trzy formaty zapisu: JSON-LD (rekomendowane), Microdata, RDFa. Google parsuje JSON-LD przez `<script type="application/ld+json">`.

Mechanika:

- Google Crawler parsuje JSON-LD podczas indeksacji
- Walidacja: <https://search.google.com/test/rich-results>
- Rich snippets w SERP: gwiazdki (Review), godziny (LocalBusiness), FAQ (FAQPage), breadcrumbs (BreadcrumbList)
- Każdy typ Schema ma swój zestaw wymaganych i opcjonalnych pól

Kod z projektu:

```
<!-- app/src/components/seo/SchemaLocalBusiness.astro -->  
---  
import { business } from '@data/business';  
  
const schema = {  
  '@context': 'https://schema.org',  
  '@type': ['LocalBusiness', 'ProfessionalService'], // multi-type  
  '@id': `${business.url}/#business`, // identyfikator do referowania w inny  
  ch schemach  
  name: business.legalName,  
  telephone: business.contact.phone,  
  email: business.contact.email,  
  address: {  
    '@type': 'PostalAddress',  
    streetAddress: business.address.street,  
    addressLocality: business.address.city,  
    postalCode: business.address.postalCode,  
    addressCountry: business.address.country,  
  },  
  geo: {  
    '@type': 'GeoCoordinates',  
    latitude: business.geo.latitude,  
    longitude: business.geo.longitude,  
  },  
  openingHoursSpecification: [  
    {  
      '@type': 'OpeningHoursSpecification',  
      dayOfWeek: ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],  
      opens: '08:00',  
      closes: '18:00',  
    },  
  ],  
}
```

```
    ],
    founder: {
      '@type': 'Person',
      name: business.founder.name,
      hasCredential: {
        '@type': 'EducationalOccupationalCredential',
        identifier: business.certificate,
      },
    },
  },
};
---
```

```
<script type="application/ld+json" set:html={JSON.stringify(schema)} />
```

Inne ważne typy schema:

- `Service` - per podstrona usługi
- `FAQPage` -> `mainEntity: [{Question, Answer}]` - generuje accordion w SERP
- `BreadcrumbList` - ścieżka okruszków pod tytułem
- `Article` / `BlogPosting` - dla bloga
- `Product` / `Offer` - dla e-commerce
- `Review` / `AggregateRating` - gwiazdki

B.3. Open Graph + Twitter Cards

Po ludzku: Ładny obrazek + tytuł + opis gdy ktoś wkleja link na Facebooku/LinkedIn/WhatsApp/Slack.

Definicja techniczna:

- **Open Graph Protocol** (Facebook, 2010) - `<meta property="og:*>`
- **Twitter Cards** (Twitter/X) - `<meta name="twitter:*>`
- Standard używany przez ~wszystkie social media + Slack/Discord/iMessage do previews

Wymagane og:

Tag	Co znaczy
<code>og:title</code>	Tytuł kartki
<code>og:description</code>	Opis
<code>og:image</code>	URL do obrazka (1200x630 px optimal)
<code>og:url</code>	Canonical URL strony
<code>og:type</code>	<code>website</code> / <code>article</code> / <code>product</code>
<code>og:locale</code>	<code>pl_PL</code>

Kod z projektu:

```
<!-- app/src/components/seo/SEO.astro -->
<meta property="og:type" content={ogType} />
<meta property="og:site_name" content="Motowycena" />
<meta property="og:title" content={fullTitle} />
<meta property="og:description" content={description} />
```

```
<meta property="og:url" content={canonicalUrl} />
<meta property="og:image" content={fullOgImage} />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
<meta property="og:locale" content="pl_PL" />

<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content={fullTitle} />
<meta name="twitter:description" content={description} />
<meta name="twitter:image" content={fullOgImage} />
```

Generowanie OG images programatycznie: Satori (Vercel) - renderuje JSX na SVG -> PNG. Można zrobić endpoint `/api/og.png?title=...`.

B.4. Canonical URL

Po ludzku: Etykieta "to jest oryginalny adres tej strony" - żeby Google nie traktował duplikatów jako konkurencji.

Definicja techniczna: `<link rel="canonical" href="...">` deklaruje preferowany URL dla danej zawartości. Rozwiązuje **duplicate content** - np. gdy ta sama treść jest pod:

- `https://example.com/article`
- `https://example.com/article/`
- `https://example.com/article?utm_source=fb`
- `https://www.example.com/article`

Bez canonical -> Google rozdziela PageRank między te warianty. Z canonical -> cały PageRank trafia do wskazanego URL.

Self-referencing canonical: każda strona ma canonical wskazujący SAMĄ SIEBIE. To NIE jest błąd - to standard.

Kod z projektu:

```
<!-- app/src/components/seo/SEO.astro -->
---
const {
  canonical = Astro.url.href, // self-referencing domyślnie
} = Astro.props;

// Force trailing slash dla zgodności z trailingSlash: 'always'
const canonicalUrl = canonical.endsWith('/') ? canonical : `${canonical}/`;
---

<link rel="canonical" href={canonicalUrl} />
```

B.5. Sitemap XML

Po ludzku: Spis treści strony dla Google - listing wszystkich URL-i z datami ostatniej modyfikacji.

Definicja techniczna: XML zgodny z sitemaps.org protokół. URL `/sitemap.xml` (lub indeks `/sitemap-index.xml` linkujący do podsitemapów). Pola:

- `<loc>` - pełny URL

- `<lastmod>` - ISO 8601 (`2026-05-21`)
- `<changefreq>` - `always` / `hourly` / `daily` / `weekly` / `monthly` / `yearly` / `never` (Google to ignoruje od ~2022)
- `<priority>` - `0.0` do `1.0` (Google też ignoruje)

Maksimum 50,000 URL i 50 MB per plik. Powyżej - używamy sitemap index.

Kod z projektu:

```
// app/astro.config.mjs
import sitemap from '@astrojs/sitemap';

export default defineConfig({
  site: 'https://www.motowycena.pl', // wymagane dla sitemap
  integrations: [
    sitemap({
      changefreq: 'monthly',
      priority: 0.7,
      lastmod: new Date(),
      serialize(item) {
        // Per-URL customization
        if (item.url === 'https://www.motowycena.pl/') {
          item.priority = 1.0;
          item.changefreq = 'weekly';
        } else if (item.url.includes('/wycena-')) {
          item.priority = 0.9;
        }
        return item;
      },
    }),
  ],
});
```

Wynik (`dist/sitemap-0.xml`):

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://www.motowycena.pl/</loc>
    <lastmod>2026-05-21T00:00:00Z</lastmod>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
  </url>
  ...
</urlset>
```

Submisja: Google Search Console -> Sitemaps ->

<https://www.motowycena.pl/sitemap-index.xml>

B.6. robots.txt

Po ludzku: Drzwi z napisem "tu nie wchodzi" dla botów Google.

Definicja techniczna: Plain-text plik w root domeny zgodny z **Robots Exclusion Protocol** (RFC 9309 od 2022, wcześniej de-facto standard od 1994). Dyrektywy dla user-agentów (botów).

Główne dyrektywy:

- `User-agent: *` - wszystkie boty
- `User-agent: Googlebot` - tylko Google
- `Disallow: /admin/` - nie indeksuj
- `Allow: /public/` - nadpisuje Disallow
- `Sitemap: https://...` - wskazuje sitemap
- `Crawl-delay: 10` - sekundy między requestami (Google ignoruje)

Pułapka: robots.txt blokuje **crawl**, nie indeksację. URL może być w indeksie z pustym snippetem jeśli ktoś go zalinkował. Do blokowania indeksacji -> `<meta name="robots" content="noindex">` lub `X-Robots-Tag` HTTP header.

Kod z projektu:

```
# app/public/robots.txt
User-agent: *
Allow: /

# Blokujemy endpointy API
Disallow: /api/

Sitemap: https://www.motowycena.pl/sitemap-index.xml
```

B.7. Trailing slash & redirect strategy

Po ludzku: Ukośnik na końcu adresu (`/kontakt/` vs `/kontakt`) - ważne dla Google, bo to dwa różne URL-e.

Definicja techniczna: HTTP traktuje `/kontakt` i `/kontakt/` jako **różne zasoby**. Bez konsystentnej strategii powstaje duplicate content. Dwie szkoły:

- 1 **Trailing slash always:** `/kontakt/` - konwencja katalogów, klasyczny WordPress
- 2 **No trailing slash:** `/kontakt` - konwencja stron API/SPA

W każdym wariantcie potrzebny jest **301 redirect** drugiego wariantu na pierwszy.

Kod z projektu:

```
// app/astro.config.mjs
export default defineConfig({
  trailingSlash: 'always', // KRYTYCZNE
  build: {
    format: 'directory', // generuje /kontakt/index.html zamiast /kontakt.html
  },
});
```

Konsekwencje:

- `dist/kontakt/index.html` (nie `dist/kontakt.html`)
- Każdy URL musi mieć `/` na końcu w canonical
- Linki wewnętrzne muszą mieć `/` : `<a href="/kontakt/">` (NIE `/kontakt`)
- Niemal każdy serwer (LiteSpeed, nginx, Apache, Cloudflare) auto-redirectuje `/kontakt` -> `/kontakt/` z 301

B.8. HTTP redirect codes (301/302/303/307/308)

Po ludzku: Różne sposoby powiedzieć "to nie tu, idź gdzie indziej".

Definicja techniczna:

Kod	Nazwa	Cache	Metoda po redirect	Kiedy używać
301	Moved Permanently	Tak	Może zmienić POST->GET	Stała zmiana URL
302	Found	Nie	Może zmienić POST->GET	Tymczasowo (legacy, unikać)
303	See Other	Nie	Zawsze GET	Po POST -> "zobacz wynik tutaj" (PRG pattern)
307	Temporary Redirect	Nie	Zachowuje metodę	Tymczasowo, force same method
308	Permanent Redirect	Tak	Zachowuje metodę	Stała zmiana, force same method

Kod z projektu:

```
// app/src/pages/api/contact.ts - PRG pattern
function success(wantsJSON: boolean): Response {
  if (wantsJSON) {
    return new Response(JSON.stringify({ success: true }), { status: 200 });
  }
  // 303 dla no-JS form POST → przekieruj na GET /kontakt/?sent=1
  return new Response(null, {
    status: 303,
    headers: { Location: '/kontakt/?sent=1' },
  });
}
```

Dlaczego 303 a nie 302:

- 302 + POST -> browser może (legacy) ponowić POST na nowy URL -> reload page = duplikat
- 303 -> MUSI zmienić na GET -> bezpieczny refresh

C. PERFORMANCE & CORE WEB VITALS

C.1. Largest Contentful Paint (LCP)

Po ludzku: Jak szybko ładuje się największa rzecz na ekranie (zwykle hero image lub heading).

Definicja techniczna: Czas od `navigationStart` do wyrenderowania największego elementu w viewporcie (image, video, block-level text). Mierzony przez Performance Observer API.

Cel Google:

- [OK] Dobre: <= 2.5s
- [!] Wymaga poprawy: 2.5s - 4.0s
- [NIE] Słabe: > 4.0s

Co optymalizować:

- 1 **Server response time** - TTFB < 600ms (Cloudflare edge to ~50ms)
- 2 **Render-blocking resources** - inline critical CSS, defer JS
- 3 **Resource load time** - preload hero image, modern formats (AVIF/WebP)
- 4 **Client-side rendering** - unikać dużych React trees blokujących main

Kod z projektu:

```
// app/astro.config.mjs
export default defineConfig({
  build: {
    inlineStylesheets: 'auto', // inline critical CSS dla LCP
  },
  prefetch: {
    prefetchAll: true, // prefetch wszystkie linki w viewport
    defaultStrategy: 'viewport', // gdy link wejdzie w viewport, prefetch zasobu
  },
  experimental: {
    clientPrerender: true, // <link rel="speculation"> w Chrome
  },
});

<!-- Preload critical resource: -->
<link rel="preload" as="image" href="/hero.webp" fetchpriority="high" />
```

C.2. Cumulative Layout Shift (CLS)

Po ludzku: Czy elementy strony "skaczą" podczas ładowania (klikasz w jedno, ładuje na innym).

Definicja techniczna: Suma wszystkich nieoczekiwanych przesunięć układu (layout shifts) podczas życia strony. Każdy shift to `impact_fraction x distance_fraction`. CLS = suma shiftów.

Cel:

- [OK] <= 0.1
- [!] 0.1 - 0.25
- [NIE] > 0.25

Główne przyczyny:

- Brak `width / height` na obrazach -> image ładuje się i pcha tekst
- Font swap (FOIT/FOUT) - tekst skacze przy zmianie czcionki
- Treść wstawiana dynamicznie (np. cookie banner) bez rezerwacji miejsca
- iframe bez `aspect-ratio`

Kod z projektu:

```
/* app/src/styles/global.css */
img {
  height: auto;
  max-width: 100%;
  /* Zawsze width/height w HTML, ale CSS daje responsive */
}

<!-- Astro <Image /> auto-dodaje width/height -->
<Image src={hero} alt="Hero" width={1200} height={600} />
```

```
/* Font-display: swap zapobiega FOIT, ale daje FOUT (lekki shift) */
/* Lepsze rozwiązanie - size-adjust matchujący fonty: */
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter.woff2') format('woff2');
  font-display: swap;
  size-adjust: 100%; /* match metric */
}
```

C.3. Interaction to Next Paint (INP)

Po ludzku: Jak szybko strona reaguje na kliknięcie/dotyk.

Definicja techniczna: Zastąpił FID (First Input Delay) w marcu 2024. Mierzy czas od **input event** (click, tap, keypress) do **next paint** (kolejny render). Bierze pod uwagę WSZYSTKIE interakcje na stronie (nie tylko pierwszą jak FID).

Cel:

- [OK] ≤ 200ms
- [!] 200ms - 500ms
- [NIE] > 500ms

Główne przyczyny słabego INP:

- Duże React trees re-renderujące się po każdej interakcji
- Synchronous JS blokujący main thread (>50ms tasks = Long Tasks)
- Brak `useMemo` / `useCallback` w hot path

Kod z projektu (jak chronimy INP w Astro):

```
<!-- Wyspy nie renderują się dopóki nie potrzeba -->
<ContactForm client:visible /> <!-- Hydrate dopiero gdy widoczne -->
<WhatsAppButton client:idle /> <!-- Hydrate gdy main thread idle -->

// React 19 - automatyczny Compiler optymalizuje memoization
// W projekcie używamy uncontrolled inputs (mniej rerenderów):
<input name="email" required /> // <-- bez useState per input
const formData = new FormData(e.currentTarget); // <-- przy submit
```

C.4. HTTP/2 i HTTP/3 (QUIC)

Po ludzku: Nowsze, szybsze protokoły przesyłania danych w internecie.

Definicja techniczna:

- **HTTP/1.1** (1997): jedno żądanie per TCP connection, head-of-line blocking
- **HTTP/2** (2015, RFC 7540): multiplexing wielu strumieni na 1 TCP, server push, header compression (HPACK)
- **HTTP/3** (2022, RFC 9114): używa **QUIC** (UDP-based) zamiast TCP - eliminuje head-of-line blocking na warstwie transportowej, szybsze handshake (0-RTT)

Mechanika QUIC:

- 1 Connection migration - ten sam connection mimo zmiany IP (np. WiFi->4G)
- 2 0-RTT handshake na drugiej wizycie - pierwszy packet zawiera już request

- 3 Encryption built-in (zawsze TLS 1.3)
- 4 Loss recovery per stream - pakiet który zaginie nie blokuje innych strumieni

Sprawdzenie:

```
curl -I --http3 https://www.motowycena.pl/  
# Server: LiteSpeed  
# Alt-Svc: h3=":443"; ma=2592000 ← deklaracja HTTP/3 support
```

Cloudflare Pages: HTTP/3 włączony domyślnie. Nic do konfiguracji.

C.5. CDN i edge caching

Po ludzku: Twoja strona istnieje w kopiach na 200+ serwerach na całym świecie. Klient z Polski dostaje kopię z Warszawy, z USA - z Nowego Jorku.

Definicja techniczna: Content Delivery Network - sieć geographically distributed reverse proxy. Cloudflare ma ~310 PoP (Points of Presence). Mechanizm:

- 1 Klient -> najbliższy PoP (anycast DNS)
- 2 PoP sprawdza cache
- 3 Hit -> zwraca z cache (< 50ms)
- 4 Miss -> fetch z origin -> cache -> serve

Cache-Control headers:

- `public, max-age=31536000, immutable` - statyczne zasoby (hashed filenames)
- `public, max-age=3600, must-revalidate` - HTML
- `private, no-cache` - per-user content

Kod z projektu:

```
# app/public/_headers (Cloudflare Pages syntax)  
  
/*  
  Strict-Transport-Security: max-age=31536000; includeSubDomains; preload  
  X-Content-Type-Options: nosniff  
  Referrer-Policy: strict-origin-when-cross-origin  
  
# Statyczne assets - long cache (immutable - hash w nazwie)  
/_astro/*  
  Cache-Control: public, max-age=31536000, immutable  
  
/fonts/*  
  Cache-Control: public, max-age=31536000, immutable
```

Astro asset hashing:

```
_astro/index.D5_KbKZ4.js    ← hash = jeśli zmieni się treść, zmieni się hash  
_astro/styles.B6Z8aFW2.css  ← long-cache OK, bo URL nigdy się nie zmieni dla danej wersji
```

C.6. Resource hints (preload/prefetch/preconnect)

Po ludzku: "Hej przeglądarko, za chwilę będę potrzebował tego pliku - pobierz go już teraz".

Definicja techniczna:


```

return (
  <form
    action="/api/contact/"    // baseline - bez JS ładuje tutaj
    method="POST"            // standard HTTP method
    onSubmit={handleSubmit}  // enhanced - tylko z JS
  >
    <input name="email" type="email" required />
    { /* HTML5 required + type validation działa BEZ JS */ }
  </form>
);
}

// Endpoint obsługuje obie ścieżki:
const contentType = request.headers.get('content-type') ?? '';
const wantsJSON = contentType.includes('application/json');

if (wantsJSON) {
  return Response.json({ success: true }); // AJAX
} else {
  return new Response(null, {                // form POST
    status: 303,
    headers: { Location: '/kontakt/?sent=1' },
  });
}

```

D.2. PRG pattern (Post/Redirect/Get)

Po ludzku: Po wysłaniu formularza -> przekierowanie -> strona "Dziękuję". Bez tego, F5 = ponowne wysłanie.

Definicja techniczna: Web design pattern (Mike Buffington, 2003) zapobiegający duplicate form submissions. Trzy fazy:

- 1 **POST** - browser wysyła form data
- 2 **303 See Other** - serwer redirectuje na URL "wynik"
- 3 **GET** - browser robi GET nowego URL, pokazuje thank-you

Bez PRG: refresh strony po POST -> browser pyta "Confirm Form Resubmission?" i ponawia POST = duplikat.

Kod z projektu:

```

// app/src/pages/api/contact.ts
function success(wantsJSON: boolean): Response {
  if (wantsJSON) {
    return new Response(JSON.stringify({ success: true }), { status: 200 });
  }
  // PRG: 303 → GET /kontakt/?sent=1
  return new Response(null, {
    status: 303,
    headers: { Location: '/kontakt/?sent=1' },
  });
}

<!-- app/src/pages/kontakt.astro -->
---
const sent = Astro.url.searchParams.get('sent') === '1';

```

```

---
{sent && (
  <div class="bg-green-50 p-4">
    Dziękuję za wiadomość!
  </div>
)}

```

D.3. MIME types form-data vs JSON

Po ludzku: Dwa różne "języki" mówienia z serwerem - jeden dla starych formularzy HTML, drugi dla nowoczesnego AJAX.

Definicja techniczna:

Content-Type	Format ciała	Kto wysyła
application/x-www-form-urlencoded	name=Jan&email=a%40b.pl	<form> POST bez enctype
multipart/form-data; boundary=...	Per-part z Content-Disposition	<form enctype="multipart/form-data"> (uploadowanie plików)
application/json	{"name": "Jan", "email": "a@b.pl"}	fetch() / XMLHttpRequest z JSON body
text/plain	Plain text	Rzadko, debug

Kod z projektu:

```

// app/src/pages/api/contact.ts - obsługa OBYDWU
const contentType = request.headers.get('content-type') ?? '';

let raw: Record<string, unknown>;
if (contentType.includes('application/json')) {
  raw = await request.json();
  // raw = { name: "Jan", email: "a@b.pl", consent: true }
} else {
  const formData = await request.formData();
  raw = Object.fromEntries(formData.entries());
  // raw = { name: "Jan", email: "a@b.pl", consent: "on" } ← checkbox value
}

```

Pułapka: checkbox w form-urlencoded zwraca "on", ale w JSON to true. Schema musi obsługiwać oba:

```

const schema = z.object({
  consent: z.union([z.literal(true), z.literal('on'), z.literal('true')]),
});

```

D.4. Honeypot anti-bot

Po ludzku: Niewidoczne pole formularza. Człowiek go nie widzi, bot wypełnia. Jak wypełnione -> bot.

Definicja techniczna: Spam detection technique. Field ukryte CSS-em (NIE display: none - bo niektóre boty to wykrywają; lepiej off-screen positioning). Pole nie ma autocomplete, ma tabIndex={-1}, aria-hidden="true". Server odrzuca submit gdy honeypot wypełniony.

Kod z projektu:

```
// app/src/components/forms/ContactForm.tsx
<div className="absolute -left-[9999px]" aria-hidden="true">
  <label>
    Strona internetowa
    <input
      type="text"
      name="website"           // "website" - nazwa atrakcyjna dla botów
      tabIndex={-1}           // klawiatura pominie
      autoComplete="off"      // password managers pominą
    />
  </label>
</div>

// Server check
if (data.website) {
  // Bot - udajemy sukces, ale nic nie wysyłamy
  return success(wantsJSON);
}
```

Schema validation:

```
const schema = z.object({
  // ...
  website: z.string().max(0).optional(), // max 0 znaków = pusty string OR brak
});
```

D.5. Cloudflare Turnstile

Po ludzku: Niewidoczny "bramkarz" sprawdzający czy klient to człowiek. Lepszy od reCAPTCHA.

Definicja techniczna: **Privacy-first CAPTCHA alternative** od Cloudflare. Używa **Private Access Tokens** (PAT) gdy dostępne (Safari/iOS), fallback do interaktywnego challenge'u. Zero cookies trackingowych. Server-side verification przez API endpoint.

Mechanika (3-step):

- 1 **Client:** `<script src="...turnstile/v0/api.js">` + `<div class="cf-turnstile" data-sitekey="...">`
- 2 **Submit:** token w form field `cf-turnstile-response`
- 3 **Server:** POST `https://challenges.cloudflare.com/turnstile/v0/siteverify` z `{secret, response: token, remoteip}`

Kod z projektu:

```
<!-- app/src/pages/kontakt.astro -->
{turnstileSiteKey && (
  <script
    src="https://challenges.cloudflare.com/turnstile/v0/api.js"
    async
    defer
  />
)}

// app/src/components/forms/ContactForm.tsx
{turnstileSiteKey && (
  <div
    className="cf-turnstile"
```

```

    data-sitekey={turnstileSiteKey}
    data-theme="light"
  />
})

// app/src/pages/api/contact.ts
async function verifyTurnstile(
  token: string,
  secret: string,
  ip: string | undefined,
): Promise<boolean> {
  const res = await fetch('https://challenges.cloudflare.com/turnstile/v0/siteverify', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({
      secret,
      response: token,
      ...(ip ? { remoteip: ip } : {}),
    }),
  });
  const json = await res.json() as { success: boolean };
  return json.success === true;
}

```

D.6. Zod runtime validation

Po ludzku: Sprawdzanie czy dane od użytkownika mają sens, niezależnie od TypeScript (TS znika przy kompilacji).

Definicja techniczna: **Schema-first validation library** dla TypeScript. Definiujesz schema -> otrzymujesz: 1) walidację runtime, 2) inferowane typy TS. Inspiracja Joi/Yup, ale type-first.

Kluczowa różnica vs TypeScript:

- TS types **znikają** przy `tsc` - nie istnieją w runtime
- Zod schema **istnieje w runtime** - sprawdza prawdziwe dane przychodzące z user input / API / DB

API:

- `schema.parse(data)` - throws na błąd
- `schema.safeParse(data)` - zwraca `{ success: true, data }` lub `{ success: false, error }`
- `z.infer<typeof schema>` - inferowany TS type

Kod z projektu:

```

// app/src/pages/api/contact.ts
import { z } from 'zod';

const schema = z.object({
  name: z.string().min(2).max(100).trim(),
  email: z.string().email().trim().toLowerCase(), // chainable transforms
  phone: z.string().min(9).max(20).trim(),
  message: z.string().min(10).max(2000).trim(),
  consent: z.union([z.literal(true), z.literal('on'), z.literal('true')]),
  website: z.string().max(0).optional(),
  'cf-turnstile-response': z.string().optional(),
});

```

```
});

type ContactPayload = z.infer<typeof schema>;
//  ^? { name: string; email: string; phone: string; message: string; consent: true | 'on' | 'true'; website?: string; ... }

const parsed = schema.safeParse(raw);
if (!parsed.success) {
  return fail(wantsJSON, 'Validation failed', 400, parsed.error.format());
}
const data = parsed.data; // typed!
```

D.7. Resend API

Po ludzku: Listonosz dla maili - serwis który przyjmuje twojego maila przez REST API i dostarcza klientowi.

Definicja techniczna: Transactional email API. Konkurencja: SendGrid, Postmark, Mailgun. Resend wyróżnia się Developer Experience - clean API, React Email templates, fair pricing (3000 free / mc).

Wymagane DNS records dla custom domain:

- **SPF** (TXT): `v=spf1 include:amazonses.com ~all`
- **DKIM** (TXT): klucz publiczny ECDSA/RSA do podpisywania
- **DMARC** (TXT): `v=DMARC1; p=quarantine; rua=mailto:dmarc@yourdomain.com`
- **MX** (opcjonalnie, dla bounce handling)

Kod z projektu:

```
// app/src/pages/api/contact.ts
async function sendEmail(data: ContactPayload, ip?: string): Promise<void> {
  const RESEND_API_KEY = import.meta.env.RESEND_API_KEY;
  const CONTACT_EMAIL = import.meta.env.CONTACT_EMAIL;

  const res = await fetch('https://api.resend.com/emails', {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${RESEND_API_KEY}`, // Bearer token auth
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      from: 'Motowycena <noreply@motowycena.pl>',
      to: [CONTACT_EMAIL],
      reply_to: data.email, // klient odpisuje bezpośrednio na maila usera
      subject: `Nowe zapytanie - ${data.name}`,
      html: `<div>...</div>`,
    }),
  });

  if (!res.ok) {
    throw new Error(`Resend API error ${res.status}`);
  }
}
```

HTML escaping (zapobiega injection):

```
function escapeHtml(s: string): string {
  return s
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}
```

E. BEZPIECZEŃSTWO I PRAWO

E.1. HTTPS, TLS i certyfikaty

Po ludzku: Zielona kłódka w pasku adresu - szyfrowane połączenie.

Definicja techniczna: HTTPS = HTTP over TLS (Transport Layer Security). TLS 1.3 (od 2018) - najnowszy standard, 1-RTT handshake. Certyfikat X.509 wystawiany przez **CA (Certificate Authority)**, weryfikowany przez chain of trust do root CA preinstalowanego w OS/browserze.

Free CA: Let's Encrypt (od 2015), ZeroSSL, Cloudflare Origin CA.

Cloudflare Pages: TLS certyfikat automatycznie z Cloudflare CA. Universal SSL. ECDSA + RSA dual-cert. TLS 1.2 + 1.3.

Sprawdzenie:

```
openssl s_client -connect motowycena.pl:443 -servername motowycena.pl < /dev/null 2>&1 | head -30
```

E.2. HSTS (Strict-Transport-Security)

Po ludzku: Mówi przeglądarce "zawsze łącz się ze mną przez HTTPS, nawet jak ktoś wpisze http://".

Definicja techniczna: HTTP header (RFC 6797). Po pierwszym poprawnym połączeniu HTTPS, browser pamięta przez `max-age` sekund i FORCE'uje HTTPS dla wszystkich przyszłych żądań.

Składnia:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
                        ↑ rok           ↑ także subdomeny  ↑ Chrome preload list
```

HSTS Preload list: hardcoded w Chrome/Firefox/Safari. Domena dodawana jednorazowo przez `https://hstspreload.org/`. Zaleca się dopiero po stabilnej konfiguracji (usunięcie domeny z preload trwa 6-12 miesięcy).

Kod z projektu:

```
# app/public/_headers
/*
  Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

E.3. Security headers (CSP, X-Frame-Options, etc.)

Po ludzku: Lista ograniczeń co przeglądarka MOŻE załadować i wyświetlić.

Definicja techniczna: HTTP response headers definiujące polityki bezpieczeństwa.

Header	Co robi
Content-Security-Policy	Whitelist źródeł: skrypty, style, obrazy, fonty
X-Frame-Options	DENY / SAMEORIGIN - clickjacking protection
X-Content-Type-Options: nosniff	Browser nie zgaduje MIME (XSS prevention)
Referrer-Policy	Co wysłać w Referer header przy linkach wychodzących
Permissions-Policy	Wyłączenie API (camera, geolocation, mic)
X-XSS-Protection	Legacy, ale jeszcze ma efekty w niektórych browserach

Kod z projektu:

```
# app/public/_headers
/*
  Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
  X-Frame-Options: SAMEORIGIN
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
  Permissions-Policy: geolocation=(), microphone=(), camera=()
  X-XSS-Protection: 1; mode=block
```

CSP (advanced):

```
Content-Security-Policy: default-src 'self';
  script-src 'self' https://challenges.cloudflare.com https://static.cloudflareinsights.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  font-src 'self';
  connect-src 'self' https://api.resend.com;
  frame-src https://challenges.cloudflare.com;
  base-uri 'self';
  form-action 'self';
```

E.4. CORS (Cross-Origin Resource Sharing)

Po ludzku: Przeglądarka pyta "czy serwer B pozwala stronie A pobierać od niego dane?". Jak nie pozwala -> blocked.

Definicja techniczna: Mechanizm browserowy ograniczający fetch między origins. **Same-Origin Policy** to baseline - tylko same protocol + same host + same port mogą się komunikować bez ograniczeń. CORS pozwala server-side wyrazić "OK, ten origin może".

Preflight request:

- 1 Browser robi `OPTIONS /endpoint` z headerami:
 - `Origin: https://example.com`
 - `Access-Control-Request-Method: POST`
 - `Access-Control-Request-Headers: content-type`
- 1 Server odpowiada:
 - `Access-Control-Allow-Origin: https://example.com`

- `Access-Control-Allow-Methods: POST, OPTIONS`
- `Access-Control-Allow-Headers: content-type`

1 Browser robi rzeczywiste POST

W naszym projekcie: same-origin (`motowycena.pl` ↔ `motowycena.pl/api/`) - CORS nie potrzebny.

Gdy potrzebne (np. API publiczne):

```
export const POST: APIRoute = async ({ request }) => {
  return new Response(/* ... */, {
    headers: {
      'Access-Control-Allow-Origin': 'https://allowed-domain.com',
      'Access-Control-Allow-Methods': 'POST, OPTIONS',
    },
  });
};
```

E.5. RODO/GDPR - legal basis i cookies

Po ludzku: Europejskie prawo o danych osobowych. Strona musi mieć politykę prywatności, cookies za zgodą, dane bezpiecznie przechowywane.

Definicja techniczna: GDPR (General Data Protection Regulation) - Regulation (EU) 2016/679, w Polsce **RODO**. Wymagania techniczne dla strony:

1 **Legal basis for processing** (Art. 6) - jedna z:

- Consent (Art. 6(1)(a)) - świadoma zgoda
- Contract (Art. 6(1)(b)) - realizacja umowy
- Legal obligation (Art. 6(1)(c))
- Vital interests (Art. 6(1)(d))
- Public task (Art. 6(1)(e))
- Legitimate interests (Art. 6(1)(f))

1 **Cookies** - PECR / ePrivacy:

- **Essential cookies** - bez zgody (session, security)
- **Analytics/Marketing** - wymagana zgoda PRZED ustawieniem

1 **Right to:**

- Access (Art. 15)
- Rectification (Art. 16)
- Erasure / "be forgotten" (Art. 17)
- Restriction (Art. 18)
- Portability (Art. 20)
- Object (Art. 21)

Co już mamy w projekcie (compliant by design):

- Cloudflare Web Analytics - **0 cookies**, IP anonimizowany
- Brak Google Analytics (wymagałby cookie consent)

- Brak Google Fonts (self-hosted Inter) - brak cross-origin requests
- Formularz: explicit consent checkbox + polityka prywatności linkowana
- Resend mail nie ustawia tracking pixeli (opcjonalne)

Kod z projektu:

```
// Explicit consent checkbox:
<label className="flex items-start gap-2">
  <input type="checkbox" name="consent" required />
  <span>
    Wyrażam zgodę na przetwarzanie moich danych osobowych w celu odpowiedzi
    na zapytanie zgodnie z{' '}
    <a href="/polityka-prywatnosci/">polityką prywatności</a>.
  </span>
</label>
```

E.6. CSRF (Cross-Site Request Forgery)

Po ludzku: Atak: złoczyńca podstawia stronę która wysyła POST w Twoim imieniu używając Twoich cookies sesji.

Definicja techniczna: Attacker oszukuje user-agent zalogowanego usera do wykonania niechcianej akcji na zaufanym serwisie. **Wymaga:** ofiara zalogowana, akcja wykonalna przez POST (lub GET z side-effectami).

Czy NASZ formularz jest podatny? Nie - nie wymaga sesji. Każdy może wysłać POST `/api/contact/`. Honeypot + Turnstile chronią przed botami, ale nie przed CSRF (bo CSRF tutaj nie ma sensu - nie ma zalogowanego usera do oszukania).

Gdy CSRF ma sens (np. panel admina):

- 1 **SameSite cookies:** `Set-Cookie: session=...; SameSite=Lax` - cookies nie wysyłane na cross-site POST
- 2 **CSRF token:** per-session unique token w form field + cookie. Server porównuje.
- 3 **Double Submit Cookie:** token w cookie + header X-CSRF-Token. Server porównuje.

Astro Actions mają built-in CSRF protection przez origin check.

F. STRATEGIA URL I LINKOWANIE

F.1. URL slug

Po ludzku: Ostatnia część adresu, np. `/wycena-celno-skarbowa/`.

Definicja techniczna: Path segment identyfikujący zasób. Konwencje:

- Małe litery
- Myślniki zamiast spacji/podkreślników (Google preferuje `-` nad `_`)
- Bez znaków diakrytycznych (`/wycena-celno-skarbowa/` zamiast `/wycena-celno-skarbówa/`)
- Krótkie ale opisowe - idealne 3-5 słów
- Stabilne - ZMIANA SLUG = utrata pozycji w Google (chyba że 301)

Kod z projektu:

```
src/pages/wycena-celno-skarbowa.astro
      |
      | to jest slug
URL → /wycena-celno-skarbowa/ (trailing slash bo config)
```

F.2. Internal linking & PageRank flow

Po ludzku: Każdy link z jednej strony na drugą przekazuje trochę "siły" SEO.

Definicja techniczna: **Internal links** dystrybuują **link equity** / **link juice** (potoczne nazwy PageRanku). Strony bliżej home (mniej kliknięć) zwykle dziedziczą więcej equity. Strategia:

- 1 **Hub & Spoke** - główna strona (**hub**) linkuje do specjalistycznych (**spokes**), spokes linkują do hub i siebie nawzajem
- 2 **Topic clusters** - grupy related contentu cross-linkujące
- 3 **Breadcrumbs** - każda podstrona ma link do parents (also good UX)

Kod z projektu:

```
<!-- app/src/components/sections/ServicePageTemplate.astro -->
<!-- "Może Cię również zainteresować" - cross-linking między usługami -->
<section>
  <h2>Może Cię również zainteresować</h2>
  {otherServices.map((s) => (
    <a href={serviceUrl(s.slug)}>
      <h3>{s.name}</h3>
      <p>{s.shortDesc}</p>
    </a>
  ))}
</section>

// app/src/data/business.ts - centralna lista linków
export const services = [...] as const;
export function serviceUrl(slug: string): string {
  return `/${slug}/`;
}
```

F.3. Hreflang (multi-language)

Po ludzku: Mówi Google "ta strona jest po polsku, jest też wersja po angielsku tu, niemiecku tu".

Definicja techniczna: `<link rel="alternate" hreflang="...">` w `<head>` lub w sitemap. Format: `language-region` (ISO 639-1 + ISO 3166-1 alpha-2).

Reguły:

- Każda strona musi linkować do SIEBIE (self-referencing hreflang)
- Wszystkie wersje muszą wzajemnie się linkować (bi-directional)
- `x-default` - fallback dla nieznanego języka

Kod (gdy dodamy EN/DE):

```
<link rel="alternate" hreflang="pl" href="https://www.motowycena.pl/wycena-celno-skarbowa/" />
<link rel="alternate" hreflang="en" href="https://www.motowycena.pl/en/customs-valuation/" />
<link rel="alternate" hreflang="de" href="https://www.motowycena.pl/de/zollbewertung/" />
```

```
<link rel="alternate" hreflang="x-default" href="https://www.motowycena.pl/wycena-celno-skarbo  
wa/" />
```

Astro support: [@astrojs/i18n](#) integration + routing.

G. HOSTING I DEPLOY

G.1. DNS i propagacja

Po ludzku: System tłumaczący `motowycena.pl` na adres IP serwera. Zmiany potrzebują czasu by dotrzeć do wszystkich.

Definicja techniczna: **Domain Name System** - hierarchiczna distributed database. Rekord DNS ma **TTL (Time To Live)** w sekundach - jak długo resolver pamięta odpowiedź.

Główne typy rekordów:

Type	Co znaczy
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias do innej domeny
MX	Mail exchange
TXT	Tekst (SPF, DMARC, weryfikacja)
NS	Nameservers
CAA	Które CA mogą wystawić cert

Strategia podczas migracji:

- 1 **Przed:** TTL 3600 (godzina)
- 2 **24h przed migracją:** obniżamy TTL do 300 (5 min)
- 3 **Migration day:** zmieniamy A/CNAME na nowy serwer
- 4 **Po stabilizacji** (kilka dni): TTL z powrotem na 3600+

Sprawdzenie:

```
dig motowycena.pl A
# motowycena.pl. 3600 IN A 192.0.2.1
#           ↑ TTL

# Propagacja globalnie:
# https://dnschecker.org/#A/motowycena.pl
```

G.2. Cloudflare Pages - deploy

Po ludzku: Wrzucasz kod do GitHuba, Cloudflare automatycznie buduje i wystawia stronę na 310 serwerach świata.

Definicja techniczna: **JAMstack hosting platform** od Cloudflare. Workflow:

- 1 Connect GitHub repo
- 2 Cloudflare clone'uje repo
- 3 Run build command (`npm run build`)
- 4 Upload `dist/` do Cloudflare edge cache (anycast network)
- 5 URL `<project>.pages.dev` + custom domain

Build config:

```
# Build settings w Cloudflare dashboard
Build command: npm run build
Build output directory: dist
Root directory: app
Environment variables:
  RESEND_API_KEY: ${SECRET}
  CONTACT_EMAIL: biuro@motowycena.pl
```

Pages Functions (serverless):

- Endpointy `/api/*` runują na **Cloudflare Workers** (V8 isolates, nie Node.js)
- Cold start: < 5ms (V8 isolates są lżejsze od Lambda containers)
- Limity: 50ms CPU per request (Free), 30s (Paid)
- API: Web Standard (`Request` , `Response` , `fetch`)

Dla naszego endpointu: potrzebujemy `@astrojs/cloudflare` adapter:

```
npm install @astrojs/cloudflare

// astro.config.mjs
import cloudflare from '@astrojs/cloudflare';

export default defineConfig({
  output: 'static', // domyślnie SSG
  adapter: cloudflare({
    mode: 'directory', // /functions/* są Workers
  }),
});
```

G.3. Edge runtime vs Node.js runtime

Po ludzku: Dwa różne "języki" runtime dla backendowych funkcji. Edge jest szybszy ale ma mniej feature'ów.

Definicja techniczna:

Cecha	Node.js	Edge (V8 isolates)
Implementacja	Node.js (libuv + V8)	V8 isolates (Cloudflare Workers, Deno Deploy)
Cold start	100-500ms	< 5ms
Memory	128 MB - 10 GB	128 MB
CPU time limit	30s+	10-50ms (free), 30s (paid)
Node APIs	<code>fs</code> , <code>crypto</code> , <code>http</code> , native modules	TYLKO web standards

Globals	<code>process</code> , <code>Buffer</code> , <code>__dirname</code>	<code>fetch</code> , <code>Request</code> , <code>Response</code> , <code>crypto.subtle</code>
Package compat	Wszystko z npm	Tylko isomorphic (no native deps)

Kod z projektu działa na Edge (sprawdź!):

```
// app/src/pages/api/contact.ts
export const POST: APIRoute = async ({ request, clientAddress }) => {
  const data = await request.json(); // web standard
  const res = await fetch('https://api.resend.com/emails', { /* */ }); //
  return new Response(JSON.stringify({...}), { //
    status: 200,
    headers: { 'Content-Type': 'application/json' },
  });
};
```

Pałapki Edge:

- [NIE] `import nodemailer` - używa Node `net` `socket`
- [NIE] `import fs` - brak filesystem
- [NIE] `process.env.X` - używaj `import.meta.env.X` lub `context.env.X`
- [OK] `fetch()` , `crypto.subtle` , `URL` , `URLSearchParams` , `Headers` , `FormData`

G.4. CI/CD przy migracji

Po ludzku: Automatyczna pipeline - commit do gita -> build -> test -> deploy.

Definicja techniczna: Continuous Integration / Continuous Deployment. W projekcie używamy **Cloudflare Pages Git integration** (zero-config CI). Alternatywy: GitHub Actions, GitLab CI, Vercel.

Typowy workflow:

```
# .github/workflows/deploy.yml (jeśli używamy GHA zamiast CF built-in)
name: Deploy

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
      - run: npm ci
      - run: npm run check          # tsc + astro check
      - run: npm run build
      - run: npm test              # gdy mamy testy
      # Deploy
      - uses: cloudflare/pages-action@v1
        with:
```

```
apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }}
accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }}
projectName: motowycena
directory: dist
```

Preview deploys:

- Każdy PR -> preview URL `<pr-id>.motowycena.pages.dev`
- Main branch -> production `motowycena.pl`
- Atomic deploys - rollback jednym kliknięciem na poprzednią wersję

H. NARZĘDZIA POMIAROWE

H.1. Lighthouse audit

Po ludzku: Google sprawdza Twoją stronę pod 4 kątami: szybkość, dostępność, SEO, "best practices".

Definicja techniczna: Open-source tool (Chrome DevTools, CLI, GitHub Action). Symuluje load na Slow 4G + Moto G4 device. Cztery kategorie:

- 1 **Performance** (40+ metrics): LCP, FCP, TBT, CLS, INP, Speed Index, Time to Interactive
- 2 **Accessibility**: ARIA, kontrast, alt texts, semantic HTML
- 3 **Best Practices**: HTTPS, console errors, deprecated APIs
- 4 **SEO**: meta, robots, structured data, mobile-friendly

CLI:

```
npm install -g lighthouse
lighthouse https://www.motowycena.pl --view --preset=desktop
```

CI integration:

```
- uses: treosh/lighthouse-ci-action@v11
  with:
    urls: |
      https://motowycena.pages.dev/
      https://motowycena.pages.dev/kontakt/
    budgetPath: ./lighthouse-budget.json
```

```
// lighthouse-budget.json
[{
  "path": "/*",
  "resourceSizes": [
    {"resourceType": "script", "budget": 100},
    {"resourceType": "total", "budget": 400}
  ],
  "timings": [
    {"metric": "interactive", "budget": 3000},
    {"metric": "largest-contentful-paint", "budget": 2500}
  ]
}]
```


H.2. Real User Monitoring (RUM) vs Lab data

Po ludzku: Lab = test w sztucznych warunkach. RUM = prawdziwe dane od prawdziwych użytkowników.

Definicja techniczna:

	Lab Data	RUM
Pomiar	Synthetic test (Lighthouse, WebPageTest)	Real users (Performance API)
Warunki	Kontrolowane (Slow 4G, fixed device)	Realne (wszystkie devices, sieci)
Próba	1 test	Tysiące userów
Reproducible	Tak	Nie
Quick feedback	Tak	Wymaga ruchu

Google używa RUM przez Chrome User Experience Report (CrUX) do rankingu Core Web Vitals.

Cloudflare Web Analytics (free RUM):

```
<!-- app/src/layouts/BaseLayout.astro -->
{cfAnalyticsToken && (
  <script
    defer
    src="https://static.cloudflareinsights.com/beacon.min.js"
    data-cf-beacon={`{"token": "${cfAnalyticsToken}"}`}
  />
)}
```

Mierzy:

- Page views, unique visitors (without cookies)
- Web Vitals (LCP, CLS, INP) per page
- Country, browser, device
- Top pages, top referrers

Alternatywy RUM: Plausible, Fathom, Umami, PostHog.

I. CONTENT MANAGEMENT

I.1. Markdown / MDX

Po ludzku: Prosty format pisania tekstu jak w notatniku, ale z formatowaniem.

Definicja techniczna:

- **Markdown** - lightweight markup (John Gruber, 2004). Sumarycznie text-to-HTML converter.
- **MDX** - Markdown + JSX. Możesz osadzać React/Astro komponenty w treści.
- **GFM (GitHub Flavored Markdown)** - rozszerzenie: tables, task lists, strikethrough, autolinks

Astro processing:

- `.md` -> parsowane przez `remark` / `rehype` plugins -> HTML
- `.mdx` -> to samo + JSX rendering

Kod z projektu:

```
---
title: Wycena celno-skarbowa
---

## Czym jest wycena celno-skarbowa?

Wycena to **dokument niezbędny** przy rejestracji pojazdu z zagranicy.

Możesz osadzać komponenty:

import CallToAction from '../..components/CTA.astro';
<CallToAction phone="509146666" />

Albo nawet React islands:

import Counter from '../..components/Counter.tsx';
<Counter client:visible initialValue={0} />
```

I.2. Headless CMS

Po ludzku: Panel administracyjny do edycji treści (jak w WordPressie), ale frontend jest osobno.

Definicja techniczna: CMS bez wbudowanego frontendu. Treść w content database/files, eksponowana przez API (REST, GraphQL). Frontend pobiera i renderuje.

Architektury:

- **Git-based** (TinaCMS, Decap CMS): treści w git, edycja przez UI -> commit
- **API-first** (Sanity, Strapi, Payload, Contentful, Hygraph): treści w DB, API
- **Hybrid** (Sanity z GROQ + frontend pobiera at build): build-time SSG z dynamic update

Porównanie dla motowycena:

Opcja	Pro	Con	Cena
MDX w gicie	Free, simple, dev-friendly	Klient musi umieć Markdown/git	\$0
TinaCMS	Visual editing, git-based	Mniej feature'ów niż Sanity	\$0 / \$39+ mc
Sanity	Pro-grade, kolaboracja	Cloud-hosted treści	\$0 / \$99+ mc
Strapi	Self-hosted, full control	Wymaga utrzymania serwera	\$0 (hosting)
Decap CMS	Free, git-based	Słaby maintenance	\$0

Co mamy w projekcie:

- Content Collections (MDX w gicie) - baseline
- Schema zod definiuje strukturę - CMS-agnostic
- Można podpiąć Tina/Sanity bez przepisywania komponentów

J. NOTACJA SEMVER I ZALEŻNOŚCI

J.1. Semantic Versioning (SemVer)

Po ludzku: Numerowanie wersji "MAJOR.MINOR.PATCH" mówiące co się zmieniło.

Definicja techniczna: Semantic Versioning 2.0.0 (semver.org). Format `X.Y.Z` :

- **MAJOR (X):** breaking changes
- **MINOR (Y):** new features, backward-compatible
- **PATCH (Z):** bug fixes, backward-compatible

Pre-release: `1.0.0-rc.1` , `1.0.0-beta.2` , `1.0.0-alpha.3` **Build metadata:** `1.0.0+20130313144700`

npm version ranges:

```
{
  "dependencies": {
    "astro": "^5.0.0",          // ^X.Y.Z = compatible with X (>=5.0.0 <6.0.0)
    "react": "~19.0.0",        // ~X.Y.Z = compatible with X.Y (>=19.0.0 <19.1.0)
    "zod": "3.24.0",           // exact
    "tailwindcss": ">=4.0.0",   // any newer
    "typescript": "*"          // any (NIE używaj na produkcji)
  }
}
```

`package-lock.json` zamraża dokładne wersje wszystkich tranzytywnych zależności.

J.2. npm vs pnpm vs yarn

Po ludzku: Trzy różne narzędzia do tego samego - instalowania paczek.

Definicja techniczna:

	npm	pnpm	yarn 4 (Berry)
Strategia store	<code>node_modules/</code> per project	Global store + hardlinks/symlinks	<code>.yarn/cache</code> (zero-installs)
Disk usage	Wysoka (duplikacja)	Niska (shared store)	Niska
Speed	Najwolniejszy	Najszybszy	Pomiędzy
Workspaces	Tak (v7+)	Tak (świetne)	Tak (świetne)
Lock file	<code>package-lock.json</code>	<code>pnpm-lock.yaml</code>	<code>yarn.lock</code>

Projekt używa npm (zero-config, każdy zna).

Ściągawka - cheatsheet komend

```
# Astro dev/build
npm run dev           # localhost:4321
npm run build         # buduje do dist/
npm run preview       # serwuje dist/ lokalnie
npm run check         # tsc + astro check (type-check)

# Astro generate types po zmianie collections
npm run astro sync

# Astro upgrade
npx @astrojs/upgrade

# Test sitemap
curl https://www.motowycena.pl/sitemap-index.xml

# Test schema markup
# https://search.google.com/test/rich-results

# Test PageSpeed
npx lighthouse https://www.motowycena.pl --view

# Sprawdzanie nagłówków
curl -I https://www.motowycena.pl/

# Test edge function (po deploy)
curl -X POST https://www.motowycena.pl/api/contact/ \
  -H "Content-Type: application/json" \
  -d '{"name":"Test","email":"test@example.com","phone":"509000000","message":"Test wiadomosc", "consent":true}'

# DNS lookup
dig motowycena.pl A
dig motowycena.pl AAAA
nslookup motowycena.pl

# SSL/TLS test
openssl s_client -connect motowycena.pl:443 -servername motowycena.pl < /dev/null
# Albo: https://www.ssllabs.com/ssltest/

# Sprawdzenie HTTP/3
curl --http3 -I https://www.motowycena.pl/
```

DODATKI

Słowniczek

10. Słowniczek pojęć

Wszystkie terminy techniczne tłumaczeniu na codzienny język - do użycia w rozmowie z klientem.

SEO	Co robi ze Google znajduje stronę. Im lepsze, tym wyższe pozycje w wyszukiwarce.
Meta description	Krotki opis pod tytułem w Google. Działa jak reklama - zachęca do kliknięcia.
Schema markup (JSON-LD)	Wizytówka strony, którą czyta Google. Mówi mu kim jesteś, gdzie działasz, co oferujesz.
Astro	Nowoczesny system do budowy stron, który generuje czysty HTML. Najszybszy dla stron usługowych.
React	System komponentów używany do interaktywnych części (formularz, przyciski). Najpopularniejszy w 2026.
Cloudflare Pages	Darmowy hosting na komputerach Cloudflare. Strona ładowana z najbliższego serwera (Polska, Niemcy, USA - automatycznie).
Sitemap (XML)	Spis treści strony dla Google - lista wszystkich podstron z datami ostatniej modyfikacji.
Trailing slash	Ukosnik na końcu adresu URL (np. /kontakt/ a nie /kontakt). Zostawiamy taki sam jak teraz - krytyczne dla SEO.
Schema LocalBusiness	Specjalna 'wizytówka' dla lokalnych firm. Po jej dodaniu Google może pokazać Twoją firmę w mapach.
Core Web Vitals	Trzy pomiary szybkości strony używane przez Google: jak szybko się ładowała, jak szybko reaguje na kliknięcie, jak stabilny jest układ.
LCP	Largest Contentful Paint - jak szybko ładowała się największa część strony. Cel: poniżej 2.5 sekundy.
CLS	Cumulative Layout Shift - czy elementy strony 'skaczą' podczas ładowania. Cel: bliski zera.
INP	Interaction to Next Paint - jak szybko strona reaguje na kliknięcie. Cel: poniżej 200ms.
OG image	Ladny obrazek, który wyświetla się gdy udostępniasz link strony na Facebooku, LinkedIn, WhatsApp.
Progressive enhancement	Formularz działa zawsze - z włączonym i wyłączonym JavaScript. Lepsze UX z JS, ale podstawowa funkcjonalność bez JS.
Cloudflare Turnstile	Niewidzialny 'bramkarz' przeciwko botom spawerskim. Lepszy niż reCAPTCHA, darmowy, zgodny z RODO.
Resend	Serwis do wysyłki maili z formularza. 3000 maili miesięcznie za darmo.
Honeypot	Niewidoczne pole w formularzu - ludzie go nie widzą, boty go wypełniają i wpadają w pułapkę.
Content Collection	Folder z plikami treści (MDX/Markdown), które Astro zamienia na strony www.

TinaCMS	Wizualny panel administracyjny do edycji treści - działa jak WordPress, ale bez WordPressa.
Sanity / Strapi	Headless CMS-y - bardziej zaawansowane panele do zarządzania treścią. Używane gdy strona ma dużo dynamicznej treści (blog, katalog).
301 redirect	Stałe przekierowanie ze starego adresu URL na nowy. My NIE potrzebujemy - bo zostawiamy wszystkie stare adresy.
Cache	Pamięć podręczna - strona zostaje 'zapamiętana' po pierwszym ładowaniu i następnym razem ładuje się błyskawicznie.
HTTPS	Zielona kłódka w pasku adresu - oznacza bezpieczne, szyfrowane połączenie. Wymagane przez Google.
CDN (Content Delivery Network)	Siec serwerów rozproszonych geograficznie. Klient z Polski łączy się z serwerem w Polsce, z Niemiec - w Niemczech. Bardzo szybko.
PRG pattern (Post/Redirect/Get)	Trzykrokový schemat dla formularzy: wysłanie, przekierowanie, pokazanie strony 'dziękuję'. Zapobiega podwójnym wysłaniom.
RODO / GDPR	Europejskie przepisy o ochronie danych osobowych. Strona musi mieć politykę prywatności, cookies tylko za zgodą, dane przechowywane bezpiecznie.
DNS	System tłumaczący nazwy domen (motowycena.pl) na adresy serwerów. Podczas wdrożenia jedyne co zmieniamy.
TTL (Time To Live)	Jak długo DNS pamięta odpowiedź. Podczas migracji obniżamy do 300s żeby szybko przełączyć serwer.